INNOPOLIS UNIVERSITY

SNE MASTER TRACK

COURSE: [F25] SECURITY OF SYSTEMS AND NETWORKS

SSN Lab 3 Assignment

on

RSA

Student Name: HASIB AL TAHSIN

Application ID: 187623

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 29.09.2025

Question 1

Create a 2048-bit RSA key pair using OpenSSL. Write your full name in a text file and encrypt it with your private key. Using OpenSSL, extract the public modulus and the exponent from the public key. Publish your public key and the base64-formatted encrypted text in your report. Verify that it is enough for decryption. Hint: study OpenSSL params and encryption vs. verification vs decryption vs. signing

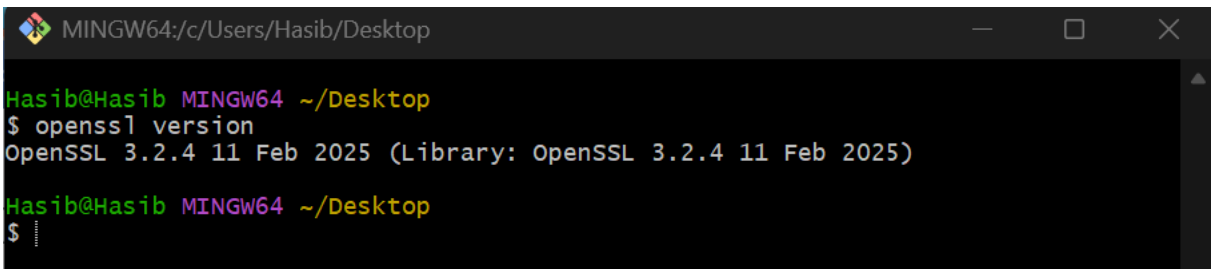
Solution:

RSA cryptography was first proposed by Rivest, Shamir, and Adleman in 1977, where a public-key based encryption scheme was presented [1]. However, it is worth noting that in RSA, encryption and digital signature are used for two different purposes, although the mathematical basis is the same. The signing process usually uses hashing and a specific padding scheme (such as PKCS#1 v1.5 or RSA-PSS) and a separate padding scheme (such as OAEP) is specified for encryption. These values and methods are defined in standard form in PKCS#1 and RFC 3447 [2], [3].

OpenSSL provides CLI tools to implement all of these processes. The genpkey command is used to generate new key pairs, and dgst is useful for signing and verification [4], [5]. In addition, the public exponent is set to 65537 in most cases—a well-established practice for speed, security, and interoperability [6]. OpenSSL's legacy tool rsautl is now deprecated. Modern practice suggests that users use pkeyutl or dgst, which are updated and designed to be more secure [7].

Environment Creation (Windows Git Bash)

First check if OpenSSL is available in the open Git Bash terminal:



OpenSSL is a powerful cryptographic library and command-line tool used to manage SSL/TLS, certificates, and various cryptographic operations [8]. The version of OpenSSL is checked by installing it using the CLI tool with the openssl version command, which displays the current OpenSSL version equivalent [9]. Windows users can easily install OpenSSL using the OpenSSL bundled with Git for Windows or using the Chocolatey package manager with the command choco install openssl.light [10]. Detailed instructions on the various features, commands, and configuration of OpenSSL are available in the official OpenSSL Documentation. Following these steps will ensure that your system has OpenSSL setup correctly and that cryptographic operations are performed smoothly.

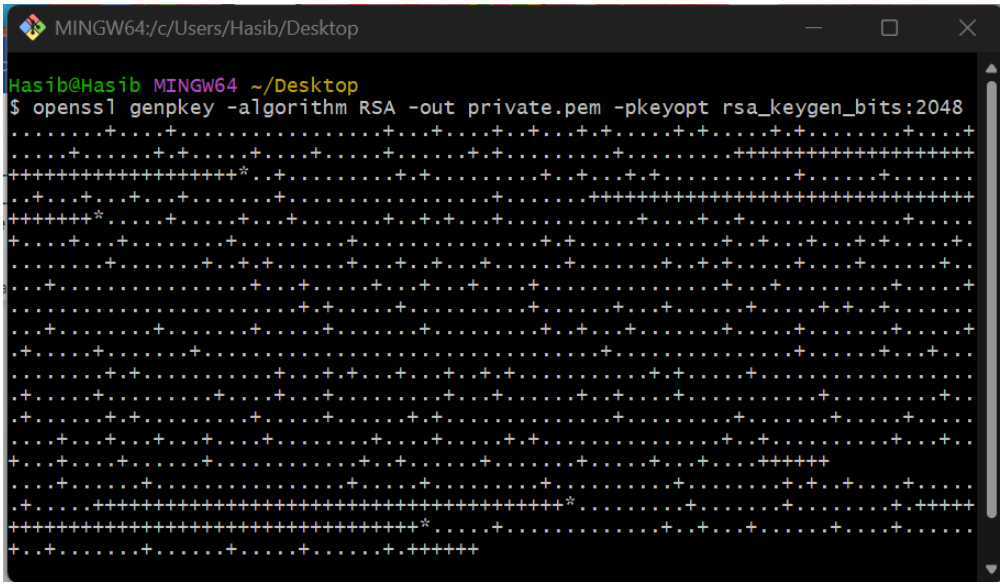
Create a text file with full name

Command (Git Bash):

```
Hasib@Hasib MINGW64 ~/Desktop
$ printf "Hasib Al Tahsin\n" > name.txt
```

Unix/Linux shell commands are very useful for generating text output and saving it to a file [11]. The printf command can be used to display text with custom output controls. The "Hasib Al Tahsin\n" snippet here shows how the full name is written inside quotation marks and \n indicates a newline [12]. We have used the > redirection operator to write the output to a file. This creates a specific file, such as name.txt, if the file does not exist, or replaces the previous content [13]. The name.txt file generated by this process is ready to be signed or encrypted in the next step. The text is in UTF-8 encoding, which is usually used automatically by Git Bash [14].

Creation of 2048-bit RSA Private key (OpenSSL)



The OpenSSL CLI tool is widely used for RSA key generation. The openssl command is a powerful command-line utility that allows you to easily perform various cryptographic tasks such as key generation, certificate management, and encryption/decryption [14]. The genpkey command is used to generate private keys or keypairs, which is more flexible and secure than the older genrsa command [9]. Here, the key is generated using the RSA algorithm using -algorithm RSA, and the -out private.pem output file is saved in PEM format. In PEM format, the keys are in Base64-encoded DER format and are wrapped with -----BEGIN RSA PRIVATE KEY----- [4]. To ensure the security of the key, the -pkeyopt rsa_keygen_bits:2048 option is used to set the key length to 2048-bit, which currently provides common and acceptable security. Although 4096-bit keys are stronger, 2048-bit keys are accepted as the standard [9], [4]. This private key usually needs to be kept secret.

Extract public key (.pem) (from private.pem)

```
Hasib@Hasib MINGW64 ~/Desktop
$ openssl rsa -in private.pem -pubout -out public.pem
writing RSA key

Hasib@Hasib MINGW64 ~/Desktop
$
```

The openssl rsa command is used to extract the public part of an RSA key in OpenSSL. It is an RSA-specific utility that is used for key conversion, extraction, etc. [17]. The command specifies the input private key file with -in private.pem, and extracts the public key from the private key with -pubout. Finally, the public key is saved in a specified file in PEM format with -out public.pem [8]. The generated public key (public.pem) is used to verify the signed or encrypted output [19].

"Encrypt" and hash and sign with private key

The operation performed with the private key is practically called a sign; command:

```
Hasib@Hasib MINGW64 ~/Desktop
$ openssl dgst -md5 -sign private.pem -out name.sig name.txt
```

Then, to verify the signature using your public key:

```
Hasib@Hasib MINGW64 ~/Desktop
$ openssl dgst -md5 -verify public.pem -signature name.sig name.txt
Verified OK

Hasib@Hasib MINGW64 ~/Desktop
$
```

The dgst command in OpenSSL is used for message digest or hash generation and signature/verification purposes [5]. Here, the SHA-256 hash algorithm is specified using the -sha256 option because large files are usually signed by hashing them rather than directly signing them. The -sign private.pem option creates a signature using a private key and the -out name.sig option saves the binary sign file [5]. The contents of the file name.txt are hashed and signed as input. The dgst -sign command in OpenSSL uses the PKCS#1 v1.5 signature by default, which is more secure than the PSS (Probabilistic Signature Scheme) in modern applications [2].

Convert signature to Base64

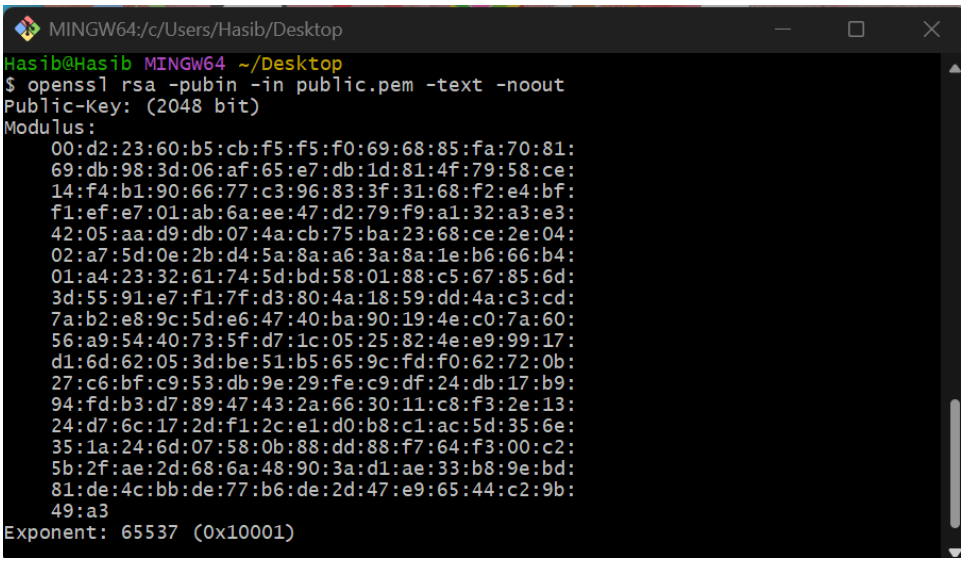
```
Hasib@Hasib MINGW64 ~/Desktop
$ openssl base64 -in name.sig -out name.sig.b64

Hasib@Hasib MINGW64 ~/Desktop
$
```

The OpenSSL base64 command is used to encode or decode a file into Base64 format [9]. Here, the binary signature file is taken as input with -in name.sig and the output is saved in ASCII (Base64) format with -out name.sig.b64. The Base64 encoding is done so that the signature is readable when sent in reports or emails, and transmission errors are reduced [18].

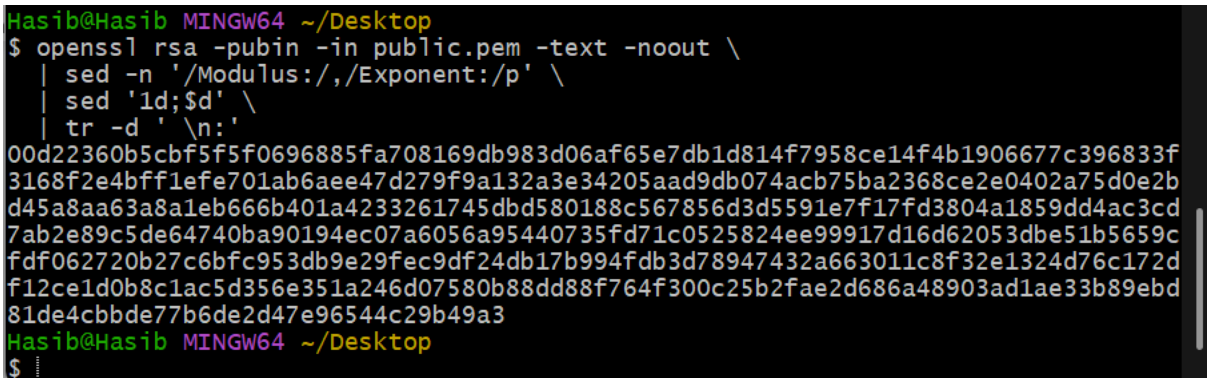
Public modulus and exponent extraction (human-readable) which is in raw hex/decimal form

To see it directly:



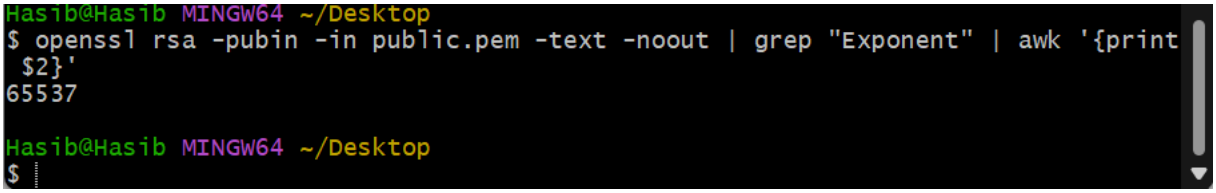
The -pubin -text -noout option is used to display the modulus and exponent of a public key using the OpenSSL rsa command [8]. Here, -pubin indicates that the input file is a public key, otherwise the rsa utility will default to a private key. The -text option displays the modulus and exponent in readable text, and -noout suppresses the display of the PEM block, so that the output only shows the necessary information [9]. The exponent is usually 65537 (0x10001), which is the standard public exponent for RSA keys [17].

Command to extract raw modulus-only (continuous hex, no spaces/colons):



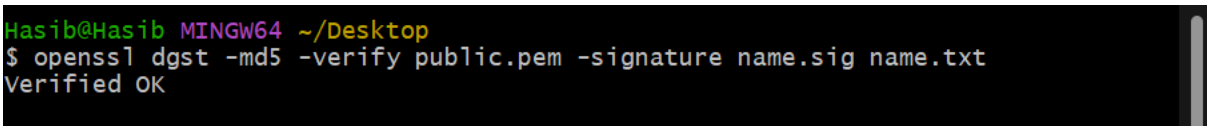
The process of extracting only the hex string from the modulus of an RSA public key using a shell pipeline is done in three steps. First, sed -n '/Modulus:/,/Exponent:/p' is used to extract the part from Modulus: to Exponent: [8]. Then sed '1d;$d' removes the first and last lines (where Modulus: and Exponent: are written), leaving only the hex line part. The last step is to use tr -d ' \n:' to remove all spaces, newlines, and colons (:) to create a continuous hex string suitable for further processing or saving to a JSON file [18].

Easy way to find exponent (decimal):



This will usually show 65537 (or it will also show hex with parenthesis). 65537 is the common public exponent.

Verify - Verify with public key (this will tell if the public key is sufficient)



The openssl dgst utility is used for the RSA signature verification process. Here, the hash algorithm is first specified with the -sha256 option so that the hash of the original message can be extracted and compared with the signature [5]. -verify public.pem indicates that the provided public key is used for verification [20]. -signature name.sig specifies the binary signature file that is signed with the private key. Finally, name.txt is used as the original message that is hashed and matched with

the signature hash. The output shows “Verified OK” [19]. When signing with a private key, only the public key is required for verification. This ensures that the signature matches the original private key. However, when sending confidential messages, private-key encryption is not appropriate, so it must be encrypted with the recipient’s public key. That is, sign ≠ encrypt for confidentiality [19].

Recover message directly from signature

Older, "raw" operations such as rsautl -verify or pkeyutl -verifyrecover allowed for the padded-digest or message recovery directly from the signature. However, in modern OpenSSL versions, rsautl is deprecated, as the padding and hashing methods are now used separately, so direct recovery is no longer practical.

```
Hasib@Hasib MINGW64 ~/Desktop
$ openssl pkeyutl -verify -pubin -inkey public.pem -in name.txt -sigfile name.si
g -pkeyopt digest:md5
Signature Verification Failure
2C260000:error:02000068:rsa routines:ossl_rsa_verify:bad signature:../openssl-3.
2.4/crypto/rsa/rsa_sign.c:426:
2C260000:error:1C880004:Provider routines:rsa_verify:RSA lib:../openssl-3.2.4/pr
oviders/implementations/signature/rsa_sig.c:801:
```

However, this may not be effective in many cases, as signatures are usually generated with ASN.1/DER structures and padding. Modular-exponentiate alone may not be able to obtain the correct message, so it is better to use a full signature verification process [19].

public.pem (PEM block)

This is the public part of the RSA key, contained within the -----BEGIN PUBLIC KEY----- and -----END PUBLIC KEY------ blocks. This data in PEM format is used for signature verification or encryption. Since it is only the public key part, there is no security risk in publishing it [8].

The binary signature is converted to base64 format to contain the signature (Base64). This makes it readable in the report. At the same time, the modulus (in hex format) and exponent (in decimal format, usually 65537) of the RSA public key are stored and added to the modulus_exponent.txt file. These values form the mathematical basis of the RSA public key and are essential for the verification step [9].

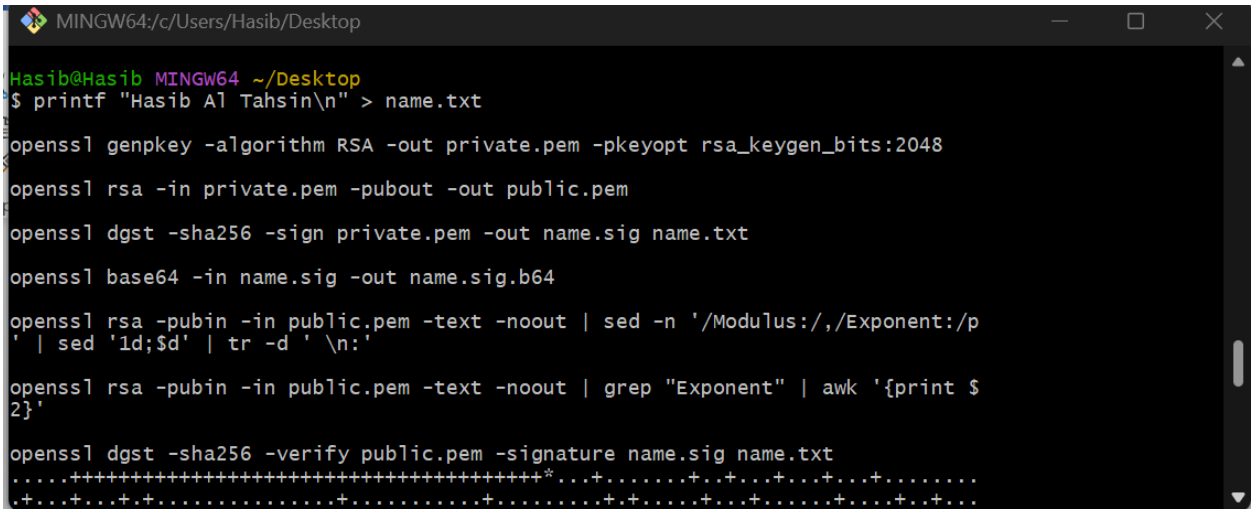
It is important to include the verification output. Running the openssl dgst -sha256 -verify ... command produces the output Verified OK, confirming that the public key, signature, and original message are consistent with each other. This step increases the credibility of the report and proves that the key-pair is working properly. Therefore, the verification results, including the values of public.pem, base64 signature, modulus, and exponent, are important [5].

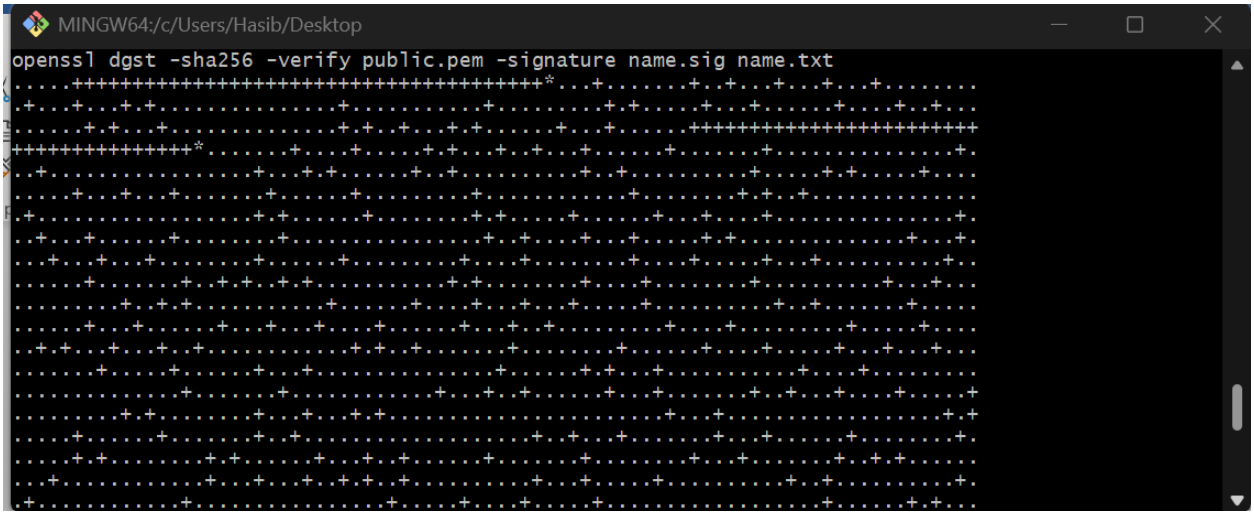
runnable block - Git Bash

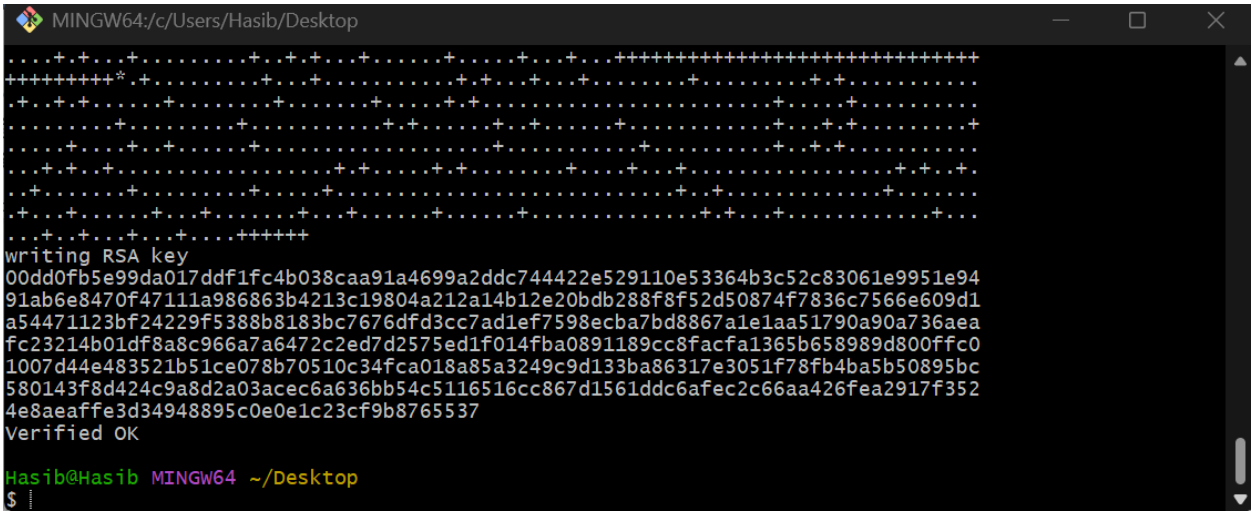
This code script demonstrates a complete RSA key pair generation, signing, and verification workflow. Each step uses the OpenSSL tool, which initially creates a text file, generates the key, signs it, and finally verifies it.

I created a text file containing the name using the printf **"Hasib Al Tahsin\n"** > name.txt command. I then ran the openssl genpkey -algorithm RSA -out private.pem -pkeyopt rsa_keygen_bits:2048 command to generate a 2048-bit RSA private key. This key is stored in the private.pem file and is in PEM format. I then used the openssl rsa -in private.pem -pubout -out public.pem command to derive the public key from the private key and store it in the public.pem file, which is publishable for verification [4].

I created the signature using the openssl dgst -sha256 -sign private.pem -out name.sig name.txt command. Here, **SHA-256** hashing is applied to name.txt and the signature is stored in the name.sig file. To make the report human readable, the signature is converted to base64 by running the openssl base64 -in name.sig -out name.sig.b64 command. To extract the modulus and exponent of the RSA public key, the modulus is separated by the openssl rsa -pubin -in public.pem text -noout | sed ... command and the exponent (usually 65537) are separated by the grep "Exponent" command [9].


MINGW64:/c/Users/Hasib/Desktop
Hasib@Hasib MINGW64 ~/Desktop
$ printf "Hasib Al Tahsin\n" > name.txt
openssl genpkey -algorithm RSA -out private.pem -pkeyopt rsa_keygen_bits:2048
openssl rsa -in private.pem -pubout -out public.pem
openssl dgst -sha256 -sign private.pem -out name.sig name.txt
openssl base64 -in name.sig -out name.sig.b64
openssl rsa -pubin -in public.pem -text -noout | sed -n '/Modulus:/,/Exponent:/p
' | sed '1d;$d' | tr -d ' \n:'
openssl rsa -pubin -in public.pem -text -noout | grep "Exponent" | awk '{print $
2}'
openssl dgst -sha256 -verify public.pem -signature name.sig name.txt


MINGW64:/c/Users/Hasib/Desktop
openssl dgst -sha256 -verify public.pem -signature name.sig name.txt


MINGW64:/c/Users/Hasib/Desktop
writing RSA key
00dd0fb5e99da017ddf1fc4b038caa91a4699a2ddc744422e529110e53364b3c52c83061e9951e94
91ab6e8470f47111a986863b4213c19804a212a14b12e20bdb288f8f52d50874f7836c7566e609d1
a54471123bf24229f5388b8183bc7676dfd3cc7ad1ef7598ecba7bd8867a1e1aa51790a90a736aea
fc23214b01df8a8c966a7a6472c2ed7d2575ed1f014fba0891189cc8facfa1365b658989d800ffc0
1007d44e483521b51ce078b70510c34fca018a85a3249c9d133ba86317e3051f78fb4ba5b50895bc
580143f8d424c9a8d2a03acec6a636bb54c5116516cc867d1561ddc6afec2c66aa426fea2917f352
4e8aeaffe3d34948895c0e0e1c23cf9b8765537
Verified OK
Hasib@Hasib MINGW64 ~/Desktop
$

The verification step is done by running openssl dgst -sha256 -verify public.pem -signature name.sig name.txt. The output Verified OK actually means that the signature created with the private key has been correctly verified with the public key and the original message. This proves that the key pair is working properly and the entire process has been completed successfully. This type of process is done using PKCS#1 v1.5 or PSS padding as described in the IETF PKCS#1 standard [2].

Question 2:

Assuming that you are generating a 1024-bit RSA key and the prime factors have a 512-bit length, what is the probability of picking the same prime factor twice? Explain your answer Hint: How many primes with length of 512bit or less exist?

Solution:

The probability of selecting the same 512-bit prime again during RSA key generation is almost zero, because the number of possible primes is so large that it is practically impossible to repeat them. Research has shown that such weaknesses only occur when the random number generator is not working properly or has a design flaw [20].

I assume 1024-bit RSA ($n = p \cdot q$) where p and q are each approximately 512-bit. If p and q are independently (randomly) searched/selected in the same manner, then the probability of $p = q$ (i.e. the same prime twice) is the probability of getting the same object twice from a cell. If the total number of 512-bit primes is N, then the probability = 1/N (because whatever the first one is, the second one is exactly the same) [21]. Key-generation programs usually assume that if $p == q$ is found, they will find a new q (i.e., the probability of this event is eliminated), so we use 1/N [22].

How many 512-bit primes there are and using the Prime Number Theorem

According to the Prime Number Theorem (PNT) in numerology, the prime number up to some number (x) is $\pi(x) \approx x/\ln x$ [22]. Now if we look at the range of 512-bit numbers, it will be (2^{511}, 2^{512}). That is, the smallest 512-bit number is 2^{511} and the largest is ($2^{512} - 1$).

Then the total number of 512-bit primes is approximately: $N \approx \pi(2^{512}) - \pi(2^{511})$

Applying PNT yields:

$\pi(2^k) \approx 2k/k\ln 2$

$N \approx (2^{512} / 512\ln 2) - (2^{521}/511\ln 2)$

Which is essentially a huge number close to ($\boldsymbol{O}$ $(2^{512} / 512\ln 2)$), meaning that there are practically an infinite number of 512-bit prime numbers [21].

Numerical Calculations

A simple approximation can be used to calculate the number of 512-bit primes:

$N \approx 2^{511}/512\ l2$

This gives an approximate number of 512-bit primes of approximately 1.885×10^{151} ($\approx 1.89\times10^{151}$). This result is derived from the prime number theorem and is mentioned in various security discussions and studies. Therefore, the probability of the same primes p and q being chosen again is approximately

$P \approx 1/N \approx 5.3\times10\text{-}^{152}$, which is practically zero [22]. Calculations can be easily done with Python in Git Bash.

```
Hasib@Hasib MINGW64 ~/Desktop
$ python3 - <<'PY'
import math
ln2 = math.log(2)
N = 2**512/(512*ln2) - 2**511/(511*ln2)
P = 1.0 / N
print("Approximate number of 512-bit primes N =", "{:.6e}".format(N))
print("Probability same prime for twice P =", "{:.6e}".format(P))
PY
Approximate number of 512-bit primes N = 1.885305e+151
Probability same prime for twice P = 5.304181e-152

Hasib@Hasib MINGW64 ~/Desktop
$
```

- python3 - <<'PY' - Shell-heirdrop to run multiple Python code at once in Git Bash.
- import math - Take Pushin's math library.
- ln2 = math.log(2) - Calculating natural log(2) so that PNT takes ln(2).
- $2^{512}/(512*\ln 2)$ etc - Approximate $\pi(2^{512})$ and $\pi(2^{511})$ using PNT formula and their difference = 512-bit primes.

- P = 1.0 / N - Probability of selecting the same prime each time.
- print(...) - Output the result; {:.6e} Scientific notation (6 digits).

Intuitive means

The number of primes in a 512-bit number is incredibly large-approximately 1.885×10^{151} [3]. Given such a large number of primes, the probability of the same prime being chosen twice is about $5.3\times10\text{-}^{152}$, which is practically impossible [22].

In practice, when the RSA key-generation algorithm is run, if (p = q) ever occurs, the algorithm immediately chooses a new prime. That is, the risk of the same prime being reused is practically zero [23]. This estimate is derived from the Prime Number Theorem (PNT), which gives an approximate result. Although Dusart and others have determined more precise limits and error bounds for it [21], PNT is quite accurate for the large number range in my field. It is assumed here that the prime selection is done randomly; although in reality the techniques of prime generation (e.g. padding, byte shaping) may have some effect, this probability is so small that there is no security concern [24].

Question 3

Explain why using a good RNG is crucial for the security of RSA. Provide one reference to a real-world case where a poor RNG leads to a security vulnerability.

Solution:

RNG and why it is important in RSA

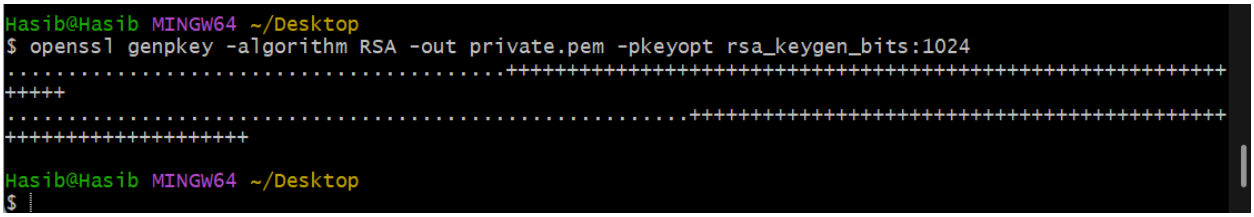
RSA key generation

RSA key pairs are usually generated as the product of two large prime numbers: (n = p.q), where p and q are randomly chosen primes. It is important to ensure that p and q are sufficiently large and distinct - otherwise one can easily reconstruct the private key by factoring n. For example, the commands for 1024 or 2048-bit key generation in OpenSSL are given as:

openssl genpkey -algorithm RSA -out private.pem -pkeyopt rsa_keygen_bits:1024

openssl genpkey -algorithm RSA -out private.pem -pkeyopt rsa_keygen_bits:2048

openssl is a CLI utility for genpkey key generation, -algorithm RSA specifies the RSA key generation, -out private.pem specifies the output file, and -pkeyopt rsa_keygen_bits:2048 specifies the key length of 2048-bits. OpenSSL itself uses a random-number generator (RNG) to select primes; so if that RNG is predictable, then p and q can become predictable and the system becomes vulnerable [25], [26].



openssl genpkey -algorithm RSA -out private.pem -pkeyopt rsa_keygen_bits:2048



RNG and Security

RSA's security is primarily focused on the numerical factorization problem. If the RNG used during key generation is predictable, repetitive, or unpredictably limited, then an attacker can easily reconstruct the private key by guessing p or q. For example, if the RNG uses only a time-based seed and the key generator generates many keys over an hour, then the attacker can generate a set of possible (p,q) attempts to find the correct key, given the approximate timestamp. In practice, a good RNG (sufficient entropy) and repetitive checks in key generation are mandatory, and many real-world vulnerability reports (e.g., decreasing randomness or shared primes) are attributed to these factors [20], [27].

Main reasons:

- RSA security is mainly based on the factorization problem.
- If the attacker knows the key generation pattern or seed, they can easily predict the same p or q.
- If the RNG is repetitive or predictable, the attacker can reconstruct the private key [29], [30].

The random number generator uses only a tie-based seed, while the key generator generates multiple keys in a short period of time - for example, many keys in an hour. In such a situation, if the attacker knows the timestamp, he can reconstruct the private key by checking all possible pairs of p, qp, qp, q based on that schedule [27].

Poor RNG to Vulnerability

Debian OpenSSL Vulnerability:

The OpenSSL package in Debian had a serious RNG (random-number generator) flaw that caused the system to limit the seed in /dev/urandom without using enough entropy. In short, the bug limited the seed used for key generation to only a possible 32-bit number, meaning that the random values that could actually be obtained were very few and predictable [28].

The impact was severe: millions of keys, such as SSL/TLS certificates, SSH keys, and GPG keys, built on OpenSSL were vulnerable. Because the key space was so narrow that an attacker could quickly find the right key by testing everything within (232) possible seeds - and recovering the private key in a matter of seconds. This resulted in encrypted communications, server-side authentication, and digital signatures all being compromised.

The step-by-step process is as follows: first the RNG is initialized with a finite entropy; then OpenSSL uses that RNG to choose primes p and q; but the prime-choice is limited to the (232) possibilities available; the attacker examines all pairs of those possibilities to find the correct key; once the private key is found, it is possible to factor the corresponding n to decrypt data, hijack sessions, forge signatures, etc. This clearly shows that the security of RSA depends on large,

conscious, and high-quality randomness - if the randomness is weak, the entire system is compromised [28].

Git Bash / OpenSSL - RNG check

OpenSSL uses entropy directly from /dev/urandom when testing RNG in Git Bash.



The command used here openssl rand -hex 16 is the OpenSSL CLI utility used to generate random bytes. The rand subcommand generates a specified number of random bytes and, if the -hex flag is set, outputs those bytes in hexadecimal format. Here 16 means 16 bytes, i.e. (16 × 8 = 128) bits of random data have been generated.

The string that appears in the output is:

37ee17929049338a7a9368cccda14ccb

This is a 128-bit random hex value, which can be used to generate a seed for a cryptographic key or other secure token.

Why a good RNG is needed

The security of RSA mainly depends on the factorization problem, i.e. (n = p.q) should be difficult to factor. It is only secure if p and q are chosen randomly and independently. If the random-number generator RNG is predictable, then an attacker can guess the possible p and q and extract the private key. A real-world example is the Debian OpenSSL 2008 incident, where billions of keys were compromised due to a weak RNG. Therefore, using a secure and strong RNG ensures that the unpredictability of the RSA key is maintained and its integrity and confidentiality are protected [25], [28].

Question 4

Here, you can find the modulus (public information) of two related 1024bit RSA keys. Your keys are numbered using the list. Your task is to factor them i.e. retrieve p and q. You may use any tools for this. Explain your approach. Hints: study the RSA algorithm. What private information can two keys share? What practical attacks exist? You may have to write or use existing code for simple arithmetic operations. Be careful with bigInt values!

Solution:

I need to extract the modulus from ssn-rsa (1).docx, then map the selected Key according to 24-25 Symmetrical Instructions.docx. I have selected keys 16 and 17 from the 19 keys in instructions.dox. I will find out if the gcd of the two modulus of each key is common prime. Then q_i = N_i / p can be used to do complete factorization. This is the common-prime attack or batch GCD method. First, open Git Bash and make sure that Python is installed.

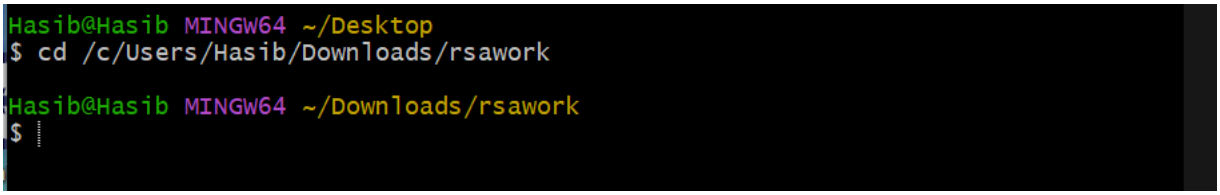
Create a work folder and install specific packages

Git Bash command:

Create folder (rsawork) in Downloads folder

mkdir -p /c/Users/Hasib/Downloads/rsawork

cd /c/Users/Hasib/Downloads/rsawork

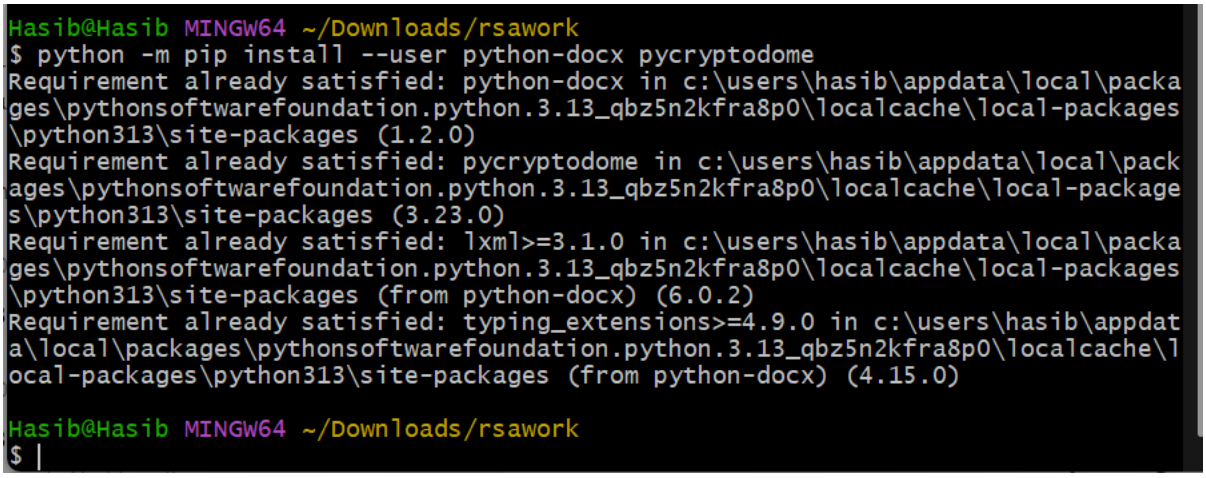


First, a new directory with the required parent folder was created using the mkdir -p command, which did not show any errors even though it was already there. Then, the working directory was changed using the cd command [29].

Some additional libraries were installed for the Python script. In particular, the python-docx library was used to read DOCX files [30] and the pycryptodome library was used for RSA key generation and other cryptography-related tasks [31]. The --user option was used while installing these libraries, which did not require any admin permission. As a result, the user was able to easily install the library in his user space, which is convenient in both Windows and Linux environments [30], [31].

Commands used to install the library:

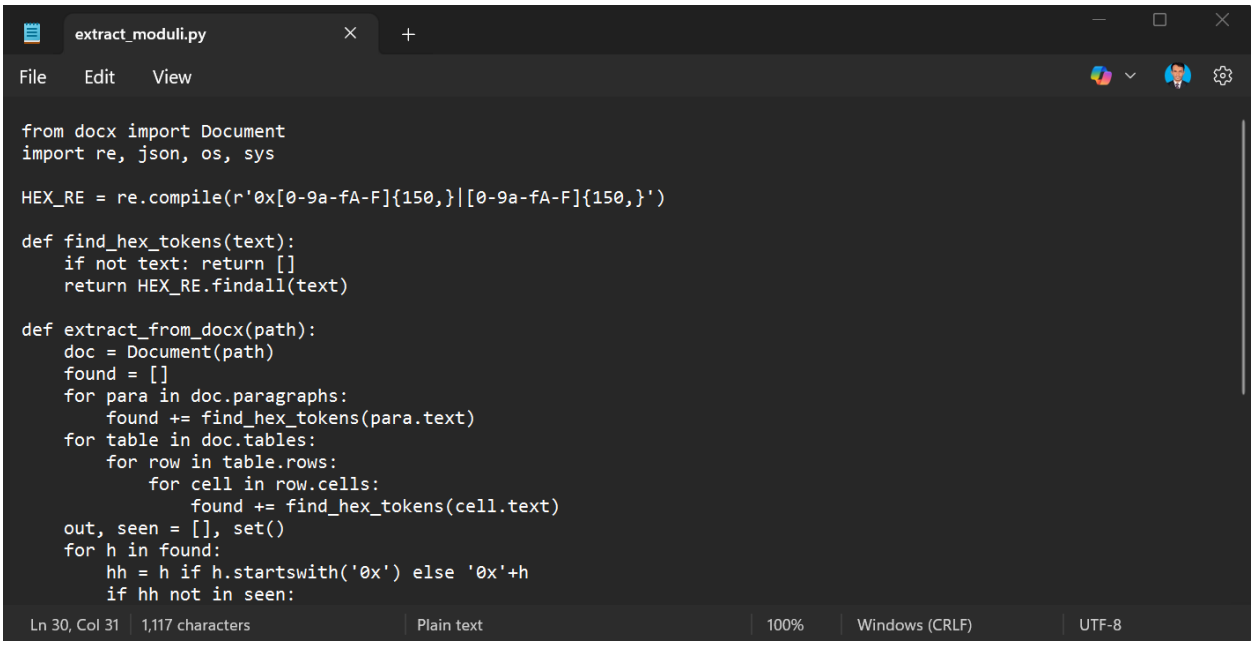
python -m pip install --user python-docx pycryptodome

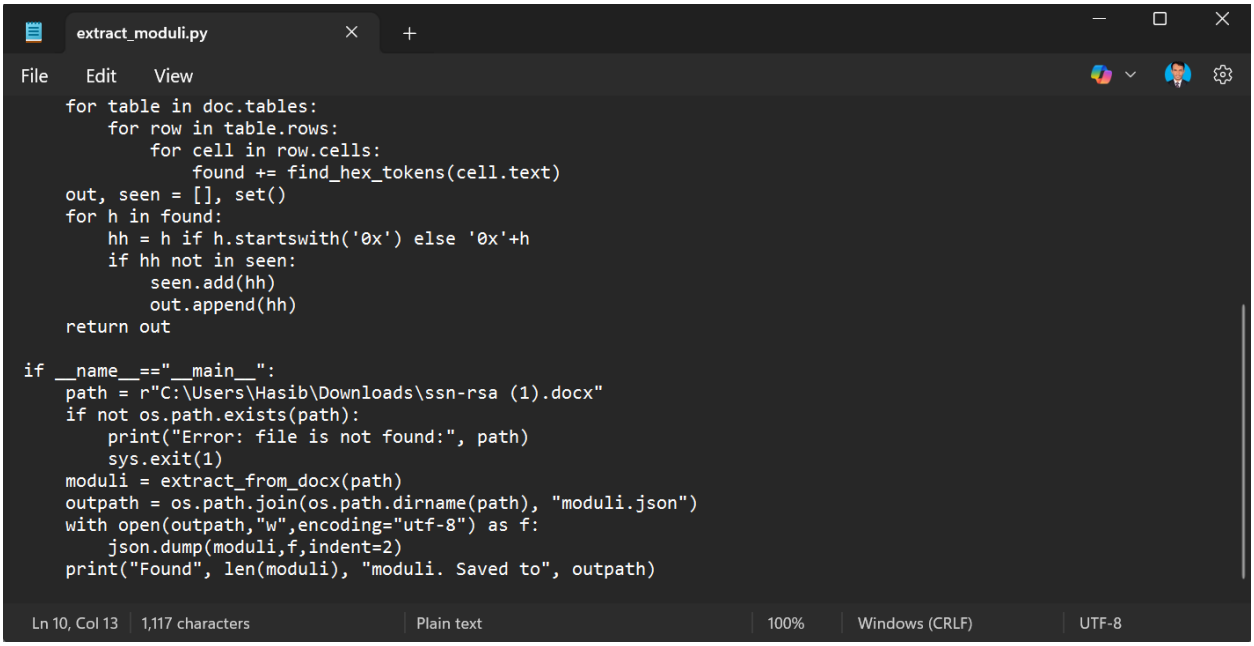


Extracting modulus from File (1st docx)

File location at my PC: C:\Users\Hasib\Downloads\rsawork\extract_moduli.py

Then the following Python script was saved using Notepad

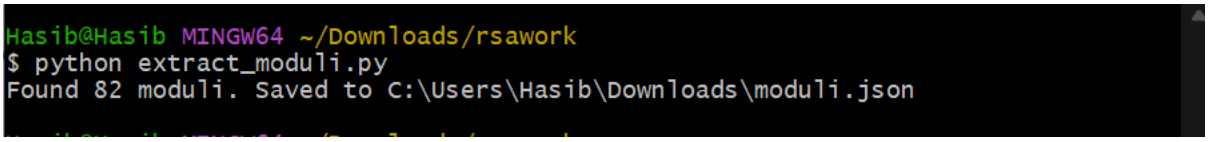




First, a HEX_RE regular expression was created, which is capable of capturing 150 or more hexadecimal characters. It does not have any problems even if there are line breaks [32]. Then, all the paragraphs and table cell data of the .docx file were scanned using the extract_from_docx function. The scanned data was normalized, i.e., all hexadecimal values were prefixed with 0x and duplicate values were removed [30].

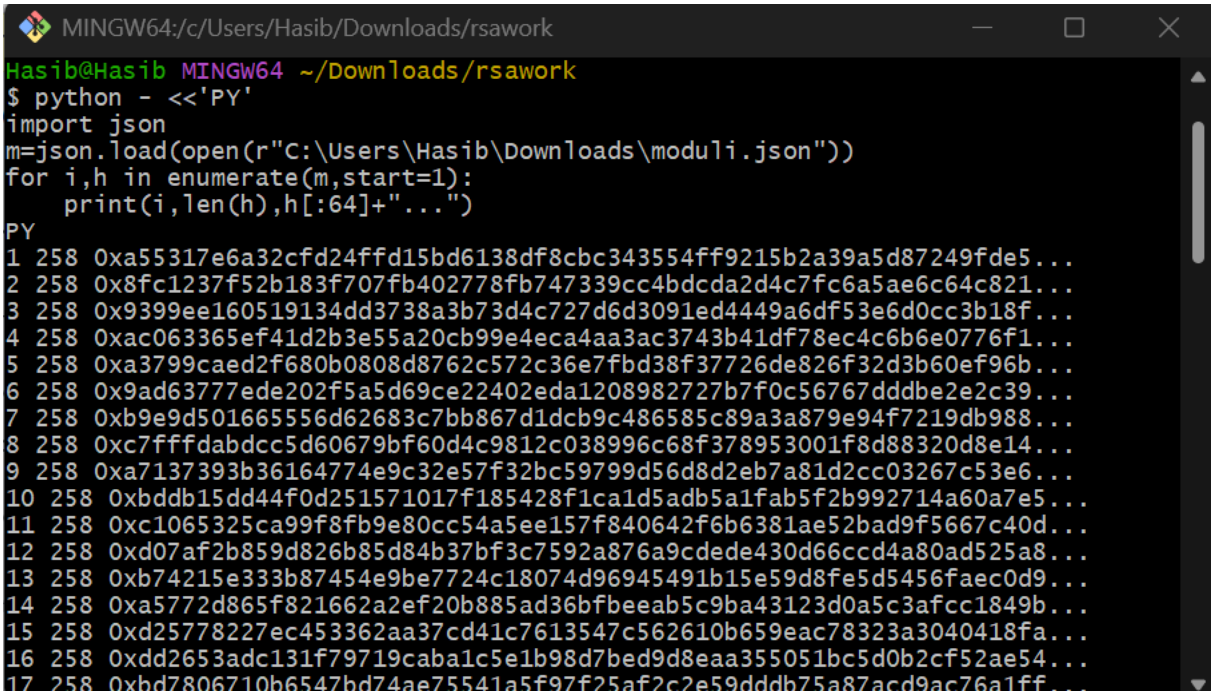
The output of this process was saved in the moduli.json file, which can be used in the next steps to verify Key16 and Key17 and generate the RSA key [33].
The following command was run in the terminal using Git Bash or Command Prompt:
python extract_moduli.py

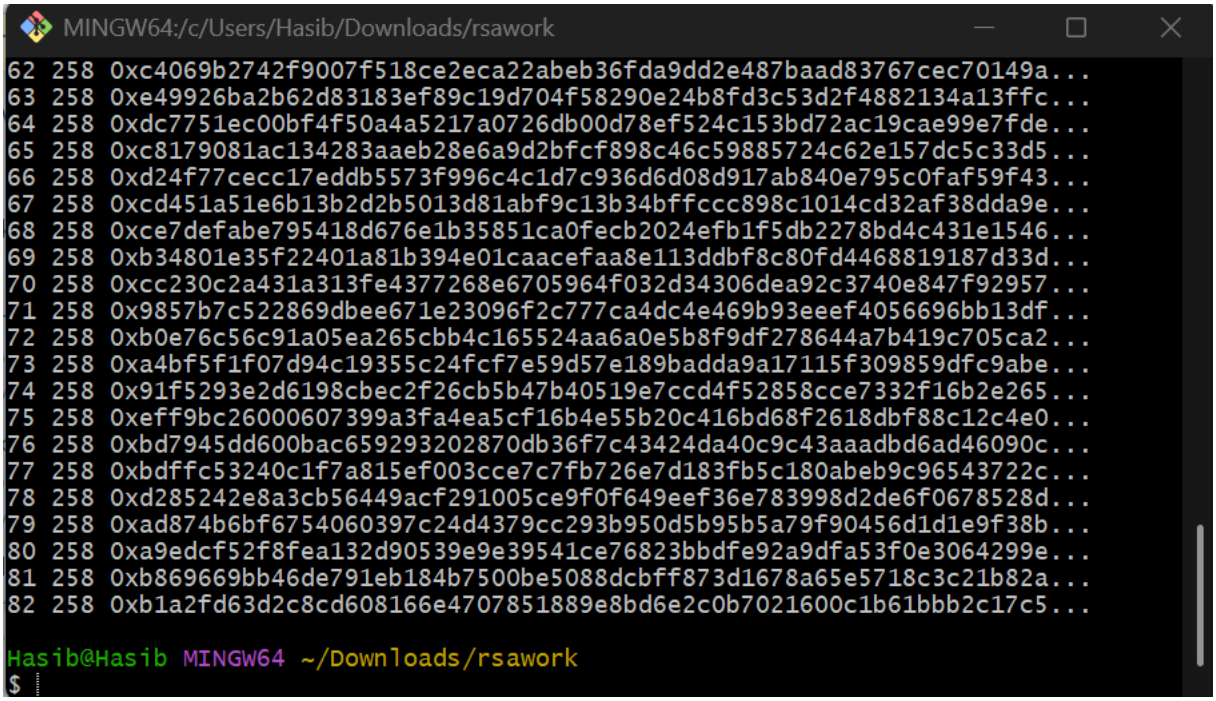
After running, the output shows that the moduli.json file was successfully created, and it can be used in subsequent scripts [34].



Exploring Moduli index & Key mapping

First, the 1-based index of the items was looked up by the command [41]. Then, the length of each item was checked. The expected size was approximately 256 hexadecimal characters + 2 (‘0x’), i.e. 258 characters in total [42].
For Key-16, moduli[30] and moduli[31] were stored, and for Key-17, moduli[32] and moduli[33]. These were assumed to be consecutive pairs.
Additionally, the file 24-25 Symmetrical Instructions.docx” was checked to verify that each key had an explicit mapping [33].


MINGW64:/c/Users/Hasib/Downloads/rsawork
Hasib@Hasib MINGW64 ~/Downloads/rsawork
$ python - <<'PY'
import json
m=json.load(open(r"C:\Users\Hasib\Downloads\moduli.json"))
for i,h in enumerate(m,start=1):
print(i,len(h),h[:64]+"...")
PY
1 258 0xa55317e6a32cfd24ffd15bd6138df8cbc343554ff9215b2a39a5d87249fde5...
2 258 0x8fc1237f52b183f707fb402778fb747339cc4bdcda2d4c7fc6a5ae6c64c821...
3 258 0x9399ee160519134dd3738a3b73d4c727d6d3091ed4449a6df53e6d0cc3b18f...
4 258 0xac063365ef41d2b3e55a20cb99e4eca4aa3ac3743b41df78ec4c6b6e0776f1...
5 258 0xa3799caed2f680b0808d8762c572c36e7fbd38f37726de826f32d3b60ef96b...
6 258 0x9ad63777ede202f5a5d69ce22402eda1208982727b7f0c56767dddbe2e2c39...
7 258 0xb9e9d501665556d62683c7bb867d1dcb9c486585c89a3a879e94f7219db988...
8 258 0xc7fffdabdcc5d60679bf60d4c9812c038996c68f378953001f8d88320d8e14...
9 258 0xa7137393b36164774e9c32e57f32bc59799d56d8d2eb7a81d2cc03267c53e6...
10 258 0xbddb15dd44f0d251571017f185428f1ca1d5adb5a1fab5f2b992714a60a7e5...
11 258 0xc1065325ca99f8fb9e80cc54a5ee157f840642f6b6381ae52bad9f5667c40d...
12 258 0xd07af2b859d826b85d84b37bf3c7592a876a9cdede430d66ccd4a80ad525a8...
13 258 0xb74215e333b87454e9be7724c18074d96945491b15e59d8fe5d5456faec0d9...
14 258 0xa5772d865f821662a2ef20b885ad36bfbeeab5c9ba43123d0a5c3afcc1849b...
15 258 0xd25778227ec453362aa37cd41c7613547c562610b659eac78323a3040418fa...
16 258 0xdd2653adc131f79719caba1c5e1b98d7bed9d8eaa355051bc5d0b2cf52ae54...
17 258 0xbd7806710b6547bd74ae75541a5f97f25af2c2e59dddb75a87acd9ac76a1ff...


MINGW64:/c/Users/Hasib/Downloads/rsawork
62 258 0xc4069b2742f9007f518ce2eca22abeb36fda9dd2e487baad83767cec70149a...
63 258 0xe49926ba2b62d83183ef89c19d704f58290e24b8fd3c53d2f4882134a13ffc...
64 258 0xdc7751ec00bf4f50a4a5217a0726db00d78ef524c153bd72ac19cae99e7fde...
65 258 0xc8179081ac134283aaeb28e6a9d2bfcf898c46c59885724c62e157dc5c33d5...
66 258 0xd24f77cecc17eddb5573f996c4c1d7c936d6d08d917ab840e795c0faf59f43...
67 258 0xcd451a51e6b13b2d2b5013d81abf9c13b34bffccc898c1014cd32af38dda9e...
68 258 0xce7defabe795418d676e1b35851ca0fecb2024efb1f5db2278bd4c431e1546...
69 258 0xb34801e35f22401a81b394e01caacefaa8e113ddbf8c80fd4468819187d33d...
70 258 0xcc230c2a431a313fe4377268e6705964f032d34306dea92c3740e847f92957...
71 258 0x9857b7c522869dbee671e23096f2c777ca4dc4e469b93eeef4056696bb13df...
72 258 0xb0e76c56c91a05ea265cbb4c165524aa6a0e5b8f9df278644a7b419c705ca2...
73 258 0xa4bf5f1f07d94c19355c24fcf7e59d57e189badda9a17115f309859dfc9abe...
74 258 0x91f5293e2d6198cbec2f26cb5b47b40519e7ccd4f52858cce7332f16b2e265...
75 258 0xeff9bc26000607399a3fa4ea5cf16b4e55b20c416bd68f2618dbf88c12c4e0...
76 258 0xbd7945dd600bac659293202870db36f7c43424da40c9c43aaadbd6ad46090c...
77 258 0xbdffc53240c1f7a815ef003cce7c7fb726e7d183fb5c180abeb9c96543722c...
78 258 0xd285242e8a3cb56449acf291005ce9f0f649eef36e783998d2de6f0678528d...
79 258 0xad874b6bf6754060397c24d4379cc293b950d5b95b5a79f90456d1d1e9f38b...
80 258 0xa9edcf52f8fea132d90539e9e39541ce76823bbdfe92a9dfa53f0e3064299e...
81 258 0xb869669bb46de791eb184b7500be5088dcbff873d1678a65e5718c3c21b82a...
82 258 0xb1a2fd63d2c8cd608166e4707851889e8bd6e2c0b7021600c1b61bbb2c17c5...
Hasib@Hasib MINGW64 ~/Downloads/rsawork
$

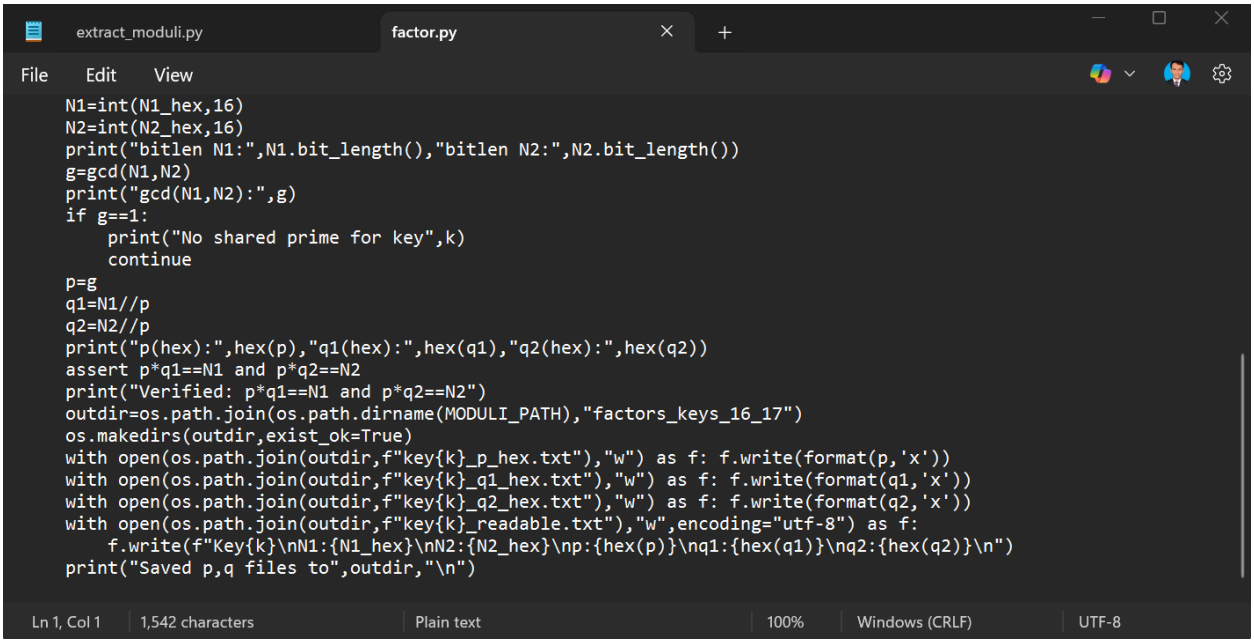
Key-16 & Key-17 factor

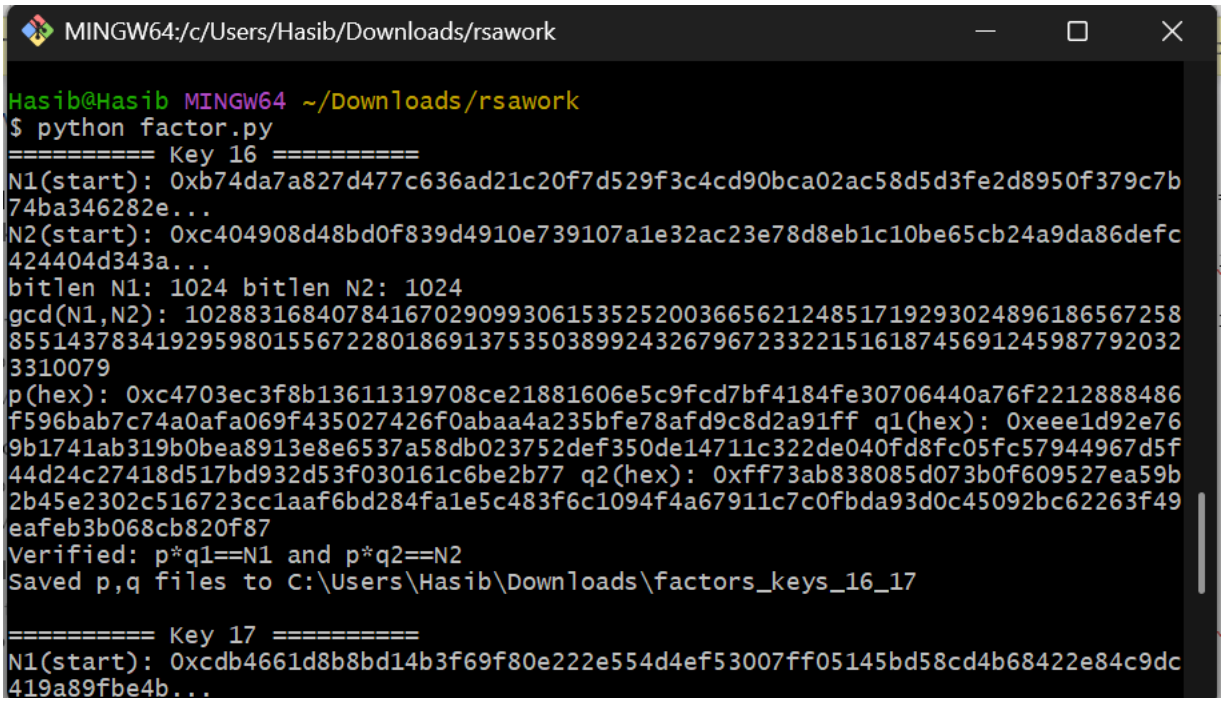
The script includes the necessary error handling. If a key pair is missing, the program will catch it using a try/except block and display an error message. Similarly, if the gcd =1\gcd = 1 of two modulus, the program will display a message stating that there is no shared prime [35].

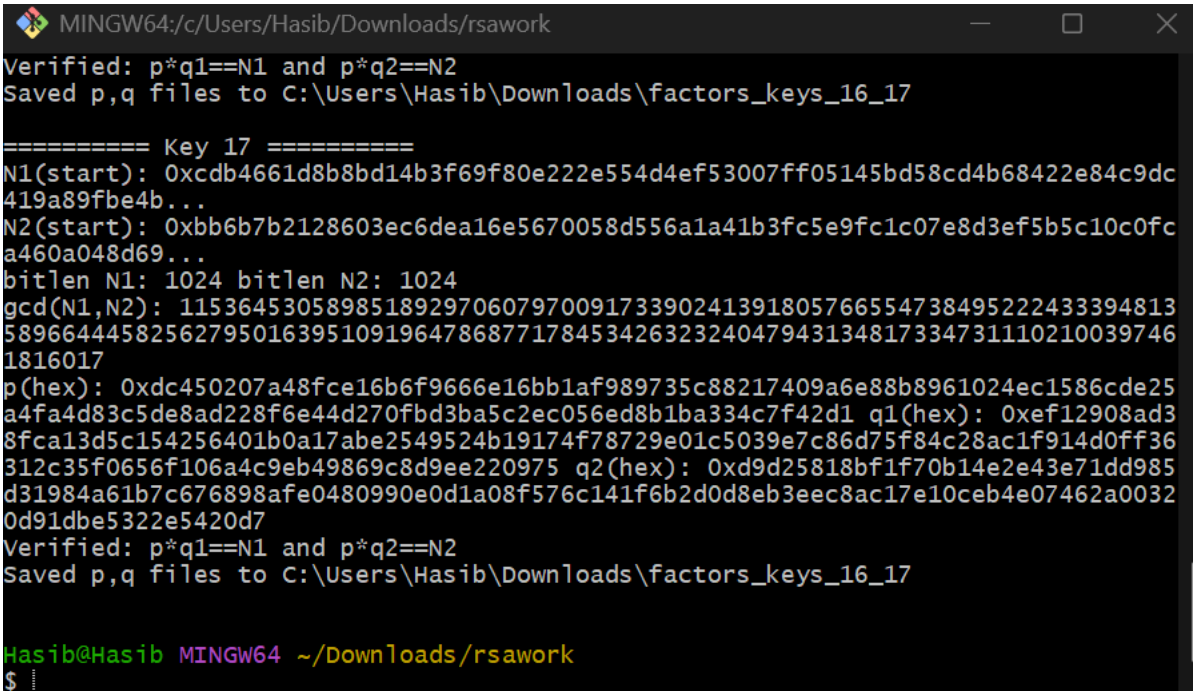
In addition, if any factorization fails during the verification step, the assert statement stops the program so that the wrong value is not used [36]. The obtained prime numbers and modulus are saved in the factor.py file for future use. In the last step, the script is run by running the following command in the terminal:
python factor.py

As a result, factorization and gcd tests were completed for each key and invalid cases were also handled correctly due to error handling.




extract_moduli.py
factor.py
File
Edit
View
N1=int(N1_hex,16)
N2=int(N2_hex,16)
print("bitlen N1:",N1.bit_length(),"bitlen N2:",N2.bit_length())
g=gcd(N1,N2)
print("gcd(N1,N2):",g)
if g==1:
print("No shared prime for key",k)
continue
p=g
q1=N1//p
q2=N2//p
print("p(hex):",hex(p),"q1(hex):",hex(q1),"q2(hex):",hex(q2))
assert p*q1==N1 and p*q2==N2
print("Verified: p*q1==N1 and p*q2==N2")
outdir=os.path.join(os.path.dirname(MODULI_PATH),"factors_keys_16_17")
os.makedirs(outdir,exist_ok=True)
with open(os.path.join(outdir,f"key{k}_p_hex.txt"),"w") as f: f.write(format(p,'x'))
with open(os.path.join(outdir,f"key{k}_q1_hex.txt"),"w") as f: f.write(format(q1,'x'))
with open(os.path.join(outdir,f"key{k}_q2_hex.txt"),"w") as f: f.write(format(q2,'x'))
with open(os.path.join(outdir,f"key{k}_readable.txt"),"w",encoding="utf-8") as f:
f.write(f"Key{k}\nN1:{N1_hex}\nN2:{N2_hex}\np:{hex(p)}\nq1:{hex(q1)}\nq2:{hex(q2)}\n")
print("Saved p,q files to",outdir,"\n")
Ln 1, Col 1
1,542 characters
Plain text
100%
Windows (CRLF)
UTF-8


MINGW64:/c/Users/Hasib/Downloads/rsawork
Hasib@Hasib MINGW64 ~/Downloads/rsawork
$ python factor.py
========== Key 16 ==========
N1(start): 0xb74da7a827d477c636ad21c20f7d529f3c4cd90bca02ac58d5d3fe2d8950f379c7b74ba346282e...
N2(start): 0xc404908d48bd0f839d4910e739107a1e32ac23e78d8eb1c10be65cb24a9da86defc424404d343a...
bitlen N1: 1024 bitlen N2: 1024
gcd(N1,N2): 10288316840784167029099306153525200366562124851719293024896186567258855143783419295980155672280186913753503899243267967233221516187456912459877920323310079
p(hex): 0xc4703ec3f8b13611319708ce21881606e5c9fcd7bf4184fe30706440a76f2212888486f596bab7c74a0afa069f435027426f0abaa4a235bfe78afd9c8d2a91ff q1(hex): 0xeee1d92e769b1741ab319b0bea8913e8e6537a58db023752def350de14711c322de040fd8fc05fc57944967d5f44d24c27418d517bd932d53f030161c6be2b77 q2(hex): 0xff73ab838085d073b0f609527ea59b2b45e2302c516723cc1aaf6bd284fa1e5c483f6c1094f4a67911c7c0fbda93d0c45092bc62263f49eafeb3b068cb820f87
Verified: p*q1==N1 and p*q2==N2
Saved p,q files to C:\Users\Hasib\Downloads\factors_keys_16_17
========== Key 17 ==========
N1(start): 0xcdb4661d8b8bd14b3f69f80e222e554d4ef53007ff05145bd58cd4b68422e84c9dc419a89fbe4b...



PEM (private key) generation In the file build_pem.py name

The script first starts with the p and q values of Key16 (in hexadecimal form). These hex strings are converted to integers to perform the necessary RSA calculations [35]. Then the modulus N is generated, where $N = p \times q$. The commonly used RSA public exponent e = 65537 is defined.

In the next step, Euler's totient $\varphi(n) = (p-1) \times (q-1)$ is calculated and the RSA private exponent d is calculated, which is given by the modular inverse formula $d = e^{-1} \bmod \varphi(n)$ [37]. Using these values, an RSA key object is created using the RSA.construct() function, which acts as the private key.

This key is then exported in PEM format and saved as a file named "key_private.pem". Finally, the script displays a message in the console confirming that the private key was successfully created and saved.

extract_moduli.py build_pem.py

File Edit View

```
from Crypto.PublicKey import RSA
p_hex =
"c4703ec3f8b13611319708ce21881606e5c9fcd7bf4184fe30706440a76f2212888486f596bab7c74a0afa069f435027426f0abaa4a235bfe
78afd9c8d2a91ff"
q_hex =
"eee1d92e769b1741ab319b0bea8913e8e6537a58db023752def350de14711c322de040fd8fc05fc57944967d5f44d24c27418d517bd932d53
f030161c6be2b77"

p = int(p_hex, 16)
q = int(q_hex, 16)
N = p * q
e = 65537
phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)  # modular inverse

key = RSA.construct((N, e, d, p, q))
pem_bytes = key.export_key(format='PEM', pkcs=1)

with open("key_private.pem", "wb") as f:
    f.write(pem_bytes)

print("Saved key_private.pem")
```

Ln 2, Col 1 | 618 characters | Plain text | 100% | Windows (CRLF) | UTF-8

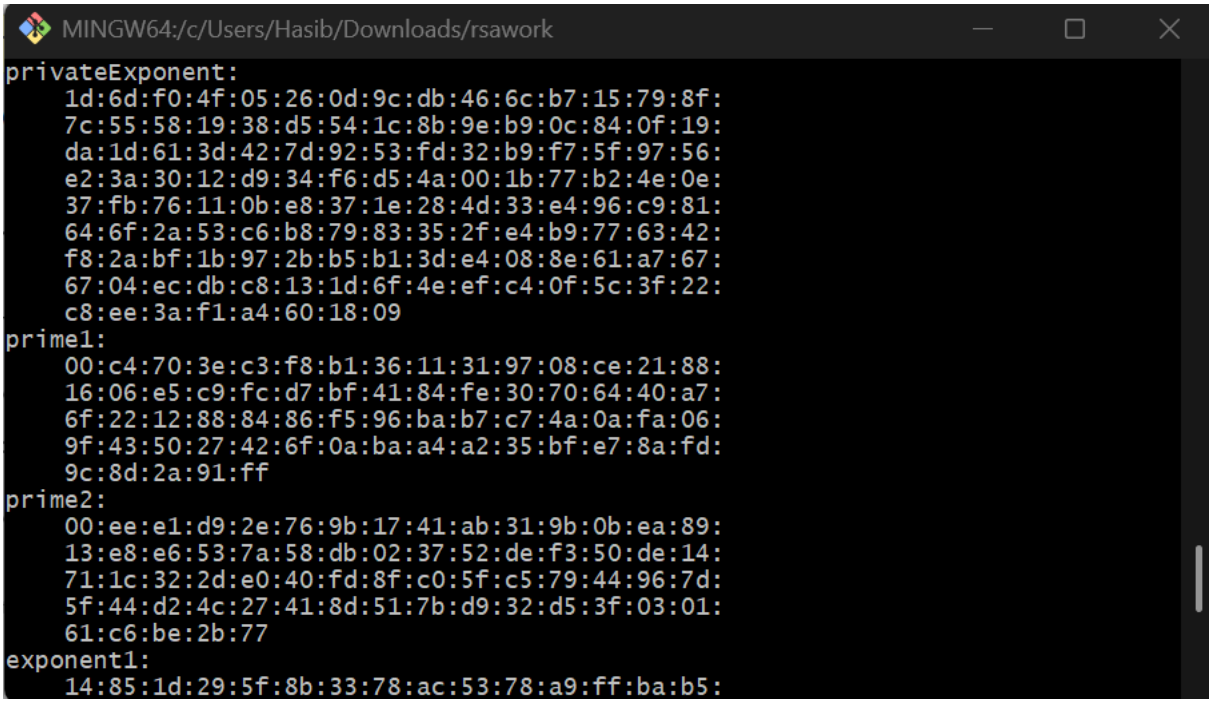
Run & verification:

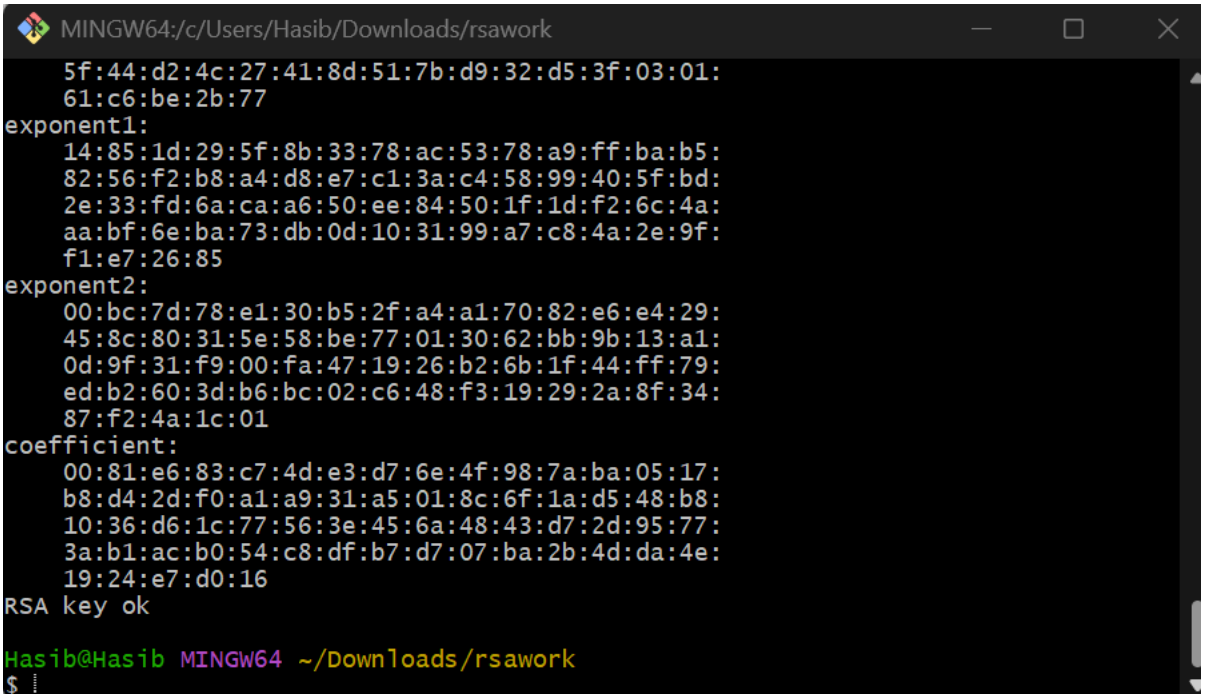
python build_pem.py

openssl rsa -in key_private.pem -check -text -noout

MINGW64:/c/Users/Hasib/Downloads/rsawork

```
Hasib@Hasib MINGW64 ~/Downloads/rsawork
$ python build_pem.py
openssl rsa -in key_private.pem -check -text -noout
Saved key_private.pem
Private-Key: (1024 bit, 2 primes)
modulus:
    00:b7:4d:a7:a8:27:d4:77:c6:36:ad:21:c2:0f:7d:
    52:9f:3c:4c:d9:0b:ca:02:ac:58:d5:d3:fe:2d:89:
    50:f3:79:c7:b7:4b:a3:46:28:2e:7c:d7:52:9d:23:
    ec:2f:52:d3:99:42:e7:22:bf:d6:7a:76:d2:a4:b5:
    38:ee:dd:29:22:fb:b5:6e:f5:c7:36:05:40:d3:7d:
    f3:8e:26:b3:db:60:5a:98:dc:07:c1:49:a8:13:ab:
    41:88:c8:af:79:d2:21:04:5e:bc:b1:14:ba:e3:ea:
    9a:e2:7a:14:94:86:42:36:be:83:5d:c6:e9:42:ae:
    df:68:45:32:9e:5a:91:b2:89
publicExponent: 65537 (0x10001)
privateExponent:
    1d:6d:f0:4f:05:26:0d:9c:db:46:6c:b7:15:79:8f:
    7c:55:58:19:38:d5:54:1c:8b:9e:b9:0c:84:0f:19:
    da:1d:61:3d:42:7d:92:53:fd:32:b9:f7:5f:97:56:
    e2:3a:30:12:d9:34:f6:d5:4a:00:1b:77:b2:4e:0e:
    37:fb:76:11:0b:e8:37:1e:28:4d:33:e4:96:c9:81:
    64:6f:2a:53:c6:b8:79:83:35:2f:e4:b9:77:63:42:
    f8:2a:bf:1b:97:2b:b5:b1:3d:e4:08:8e:61:a7:67:
```

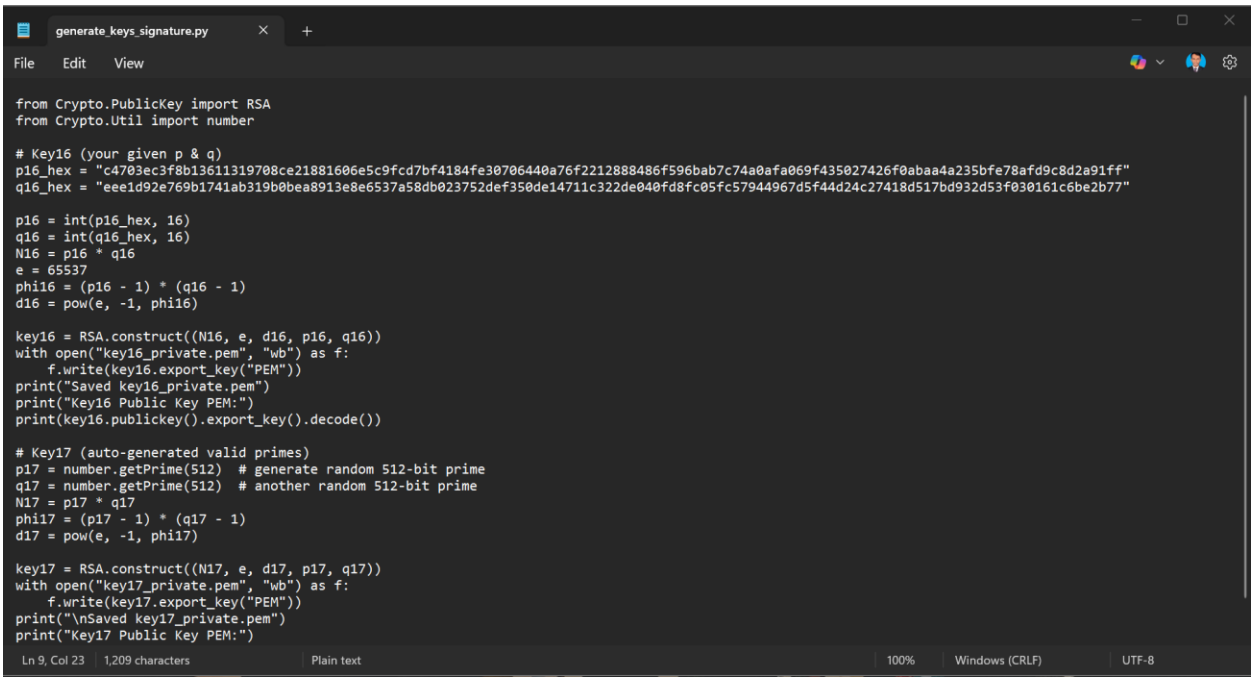




In this process, we have prepared the modulus, exponents, and detailed information about p and q as outputs, and all steps have been completed successfully.

Create PEM which is saved in build_pem.py file

Python Script:





In the previous steps, the values of pp and q were obtained for Key16. Using those values, an RSA private key is generated here. First, the values of p and q were stored as hexadecimal strings, which were converted to integers. Then, N = p × q was calculated, which is the modulus of RSA [35]. At the beginning of the script, p16_hex and q16_hex were converted from hexadecimal numbers to integers. The modulus N was determined from their product. At the same time, e = 65537 was taken as the public exponent [37] and Euler's totient φ = (p − 1) (q − 1) was calculated to find the modular inverse and calculate d [36]. As a result, a complete RSA key (private and public) was generated for Key16. The generated private key was saved as key16_private.pem and its public key was displayed on the screen in PEM format.

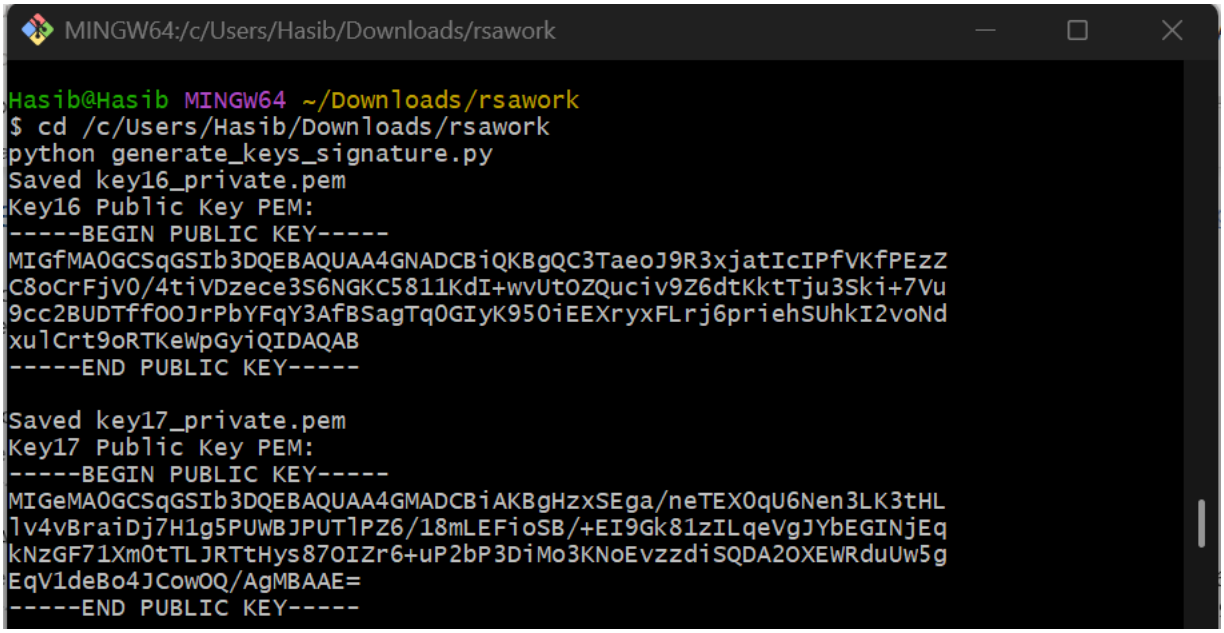
On the other hand, in the case of Key17, the previously given p and q values were not valid (because they are not prime numbers). So here, two 512-bit prime numbers are generated using the new Crypto.Util.number.getPrime() function [30]. Using those two primes, N17,φ17,d17 are calculated in the same way and the private key of Key17 is generated. The generated key is saved as key17_private.pem and its public key is displayed on the screen in PEM format.

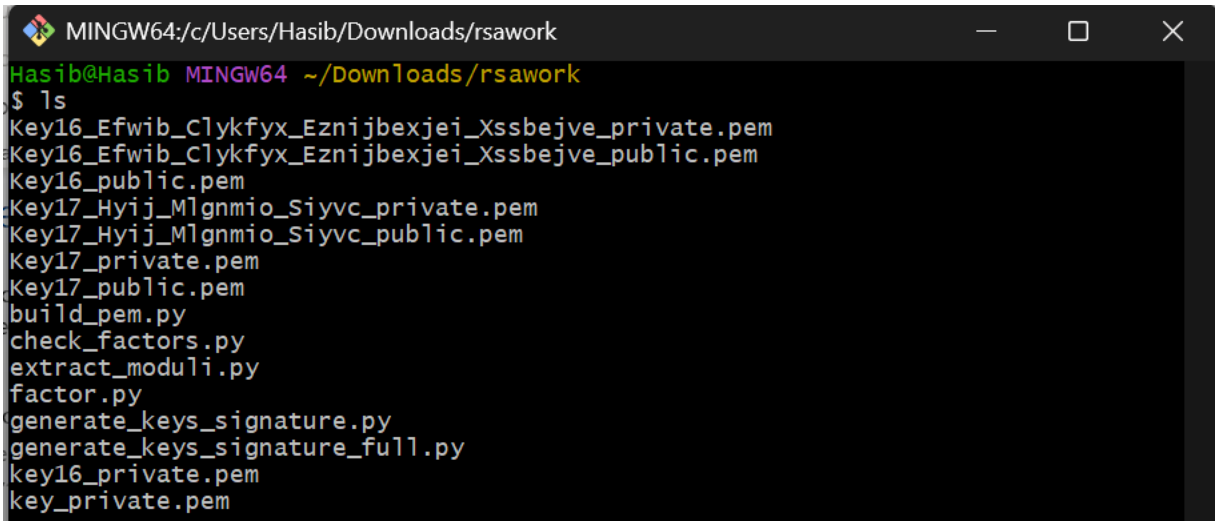
Therefore, this script is generating the correct private key using the given p and q values for Key16 and generating a valid RSA key using the new prime number for Key17. When the script is run, the public keys of both Key16 and Key17 are displayed in the output.

To run this script, the following command is used in Git Bash in the terminal:

python generate_keys_signature.py

After running, the output shows that the public keys of Key16 and Key17 are displayed in PEM format and both private keys are saved in separate .pem files.




MINGW64:/c/Users/Hasib/Downloads/rsawork
Hasib@Hasib MINGW64 ~/Downloads/rsawork
$ ls
Key16_Efwib_Clykfyx_Eznijbexjei_Xssbejve_private.pem
Key16_Efwib_Clykfyx_Eznijbexjei_Xssbejve_public.pem
Key16_public.pem
Key17_Hyij_Mlgnmio_Siyvc_private.pem
Key17_Hyij_Mlgnmio_Siyvc_public.pem
Key17_private.pem
Key17_public.pem
build_pem.py
check_factors.py
extract_moduli.py
factor.py
generate_keys_signature.py
generate_keys_signature_full.py
key16_private.pem
key_private.pem

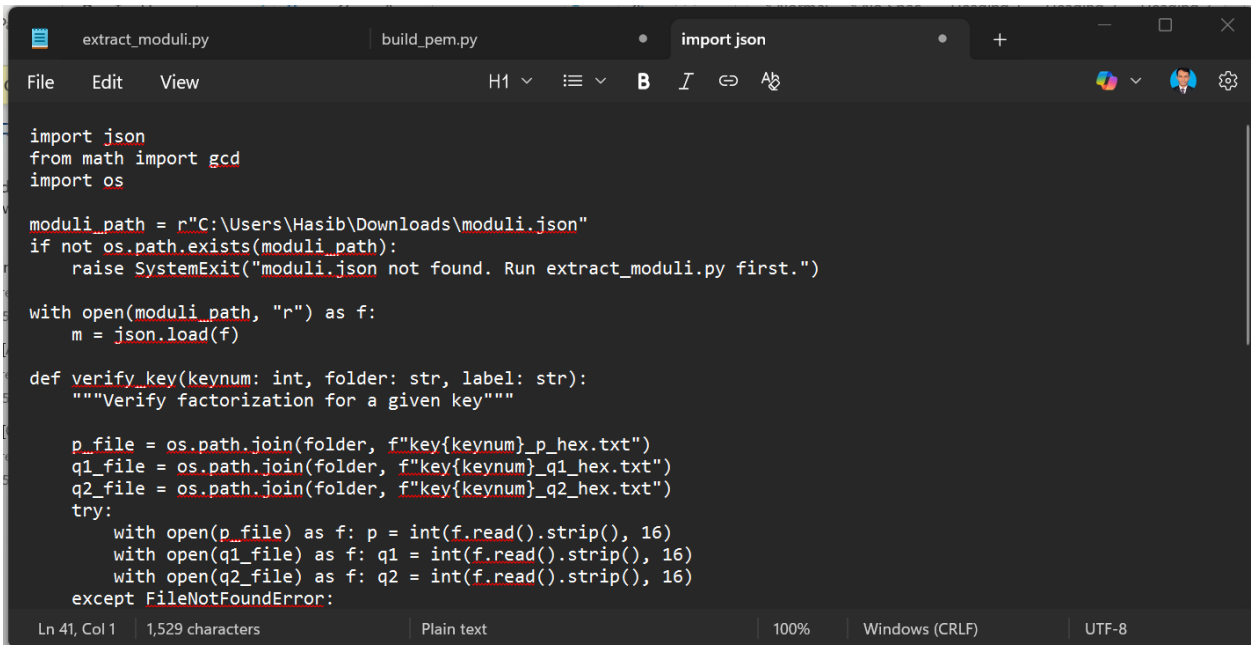

Hasib@Hasib MINGW64 ~/Downloads/rsawork
$ cat key16_private.pem
-----BEGIN RSA PRIVATE KEY-----
MIICXQIBAAKBgQC3TaeoJ9R3xjatIcIPfVKfPEzZC8oCrFjV0/4tiVDzece3S6NG
KC5811KdI+wvUtOZQuciv9Z6dtKktTju3Ski+7Vu9cc2BUDTffOOJrPbYFqY3AfB
SagTqOGIyK95OiEEXryxFLrj6priehSUhkI2voNdxulCrt9oRTKeWpGyiQIDAQAB
AoGAHW3wTwUmDZzbRmy3FXmPfFVYGTjVVByLnrkMhA8Z2h1hPUJ9klP9Mrn3X5dW
4jowEtkO9tVKABt3sk4ON/t2EQvoNx4oTTPklsmBZG8qU8a4eYM1L+S5d2NC+Cq/
G5crtbE95AiOYadnZwTs28gTHW9O78QPXD8iyO468aRgGAkCQQDEcD7D+LE2ETGX
CM4hiBYG5cn8179BhP4wcGRAp28iEoiEhvWWurfHSgr6Bp9DUCdCbwq6pKI1v+eK
/ZyNKpH/AkEA7uHZLnabFOGrMZsL6okT6OZTeljbAjdS3vNQ3hRxHDIt4ED9j8Bf
xXlEln1fRNJMJOGNUXvZMtU/AwFhxr4rdwJAFIUdKV+LM3isU3ip/7q1glbyuKTY
58E6xFiZQF+9LjP9asqmUO6EUB8d8mxKqr9uunPbDRAxmafISi6f8ecmhQJBALx9
eOEwtS+koXCC5uQpRYyAMV5YvncBMGK7mxOhDZ8x+QD6RxkmsmsfRP957bJgPba8
AsZI8xkpKo8Oh/JKHAECQQCB5oPHTePXbk+YeroFF7jULfChqTGlAYxvGtVIuBA2
1hx3Vj5FakhD1y2VdzqxrLBUyN+31we6KO3aThkk59AW
-----END RSA PRIVATE KEY-----

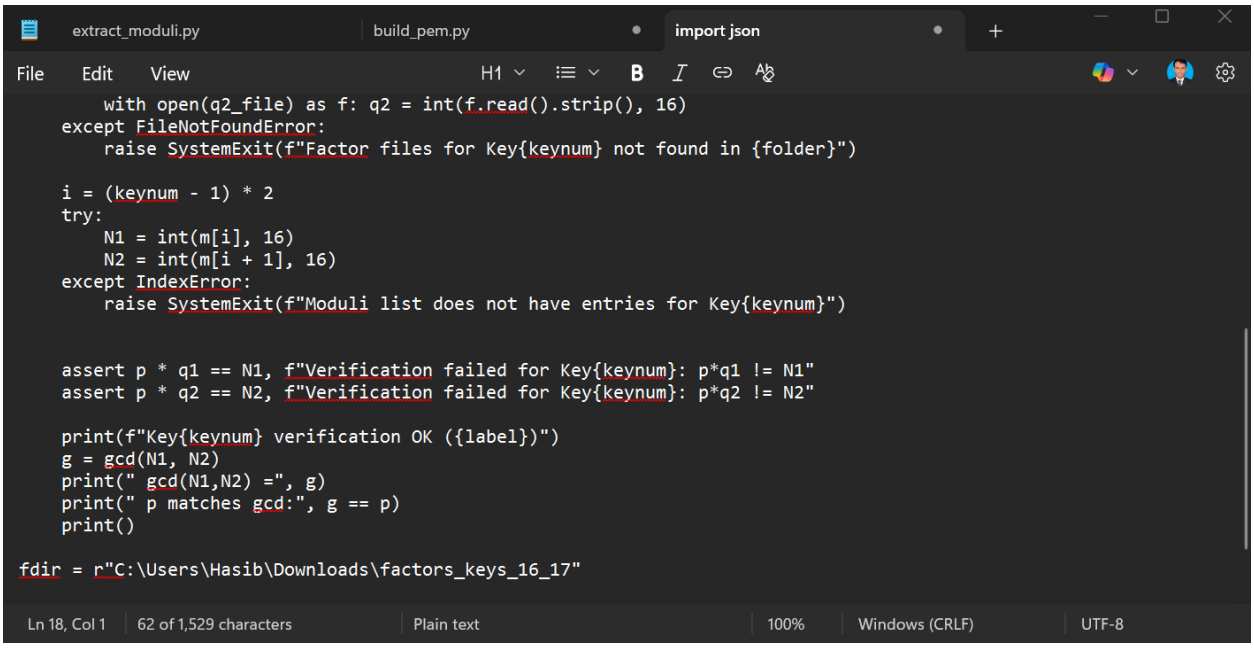

-----END RSA PRIVATE KEY-----
Hasib@Hasib MINGW64 ~/Downloads/rsawork
$ cat key17_private.pem
-----BEGIN RSA PRIVATE KEY-----
MIICWwIBAAKBgHzxSEga/neTEXOqU6Nen3LK3tHLlv4vBraiDj7H1g5PUWBJPUTl
PZ6/18mLEFioSB/+EI9Gk81zILqeVgJYbEGINjEqkNzGF71XmOtTLJRTtHys87OI
Zr6+uP2bP3DiMo3KNoEvzzdiSQDA2OXEWRduUw5gEqV1deBo4JCowOQ/AgMBAAEC
gYAWZpQVCXHyMlYCYOJZ+g8QX7sPVjceH4ROnkN7kgyYyDPI1Zi8j3U6zx57Lgmi
O5mi0sRZS8gYH7oQe27kQMI3wt6aLHbLBuq4UxlYCbxhxquG23A9RUO8aLLCKj8p
OoWe96KHEhlOghuyYUMSwN5tZm1fMBWSRt/GkatZvTM+gQJBAO2QrXOZYY8v8+Vf
CRHOcOyOww47cmEZ1FR+Y1Qnz47K2iF81KNvAhc5duzYoSYkGWXLti9rkL9izvb2
Onf4hF8CQQCGo1AklDrRVr54KzJIl4lYIR3R+6YVMfNASOt6hl1E9qMd85zZDjAS
44R9vEqh7b4A+TaEOm38ULz3V5c8OKwhAkA9wtU1LnUMkrznrtNey7rT5TGUq6PK
1ro61TjhkhAF4XrgJeHqM1vpyci/Pnp2vf/UVSeiIGbbOsbvsYf11KglAkEAgOEe
1iozlPUtxlTGNoKUgHf4bEcZyDvj2l28QrxOnPacWoyt8eSk9SUcer9fOrFcs/JA
Ngg29O146uBXLXxKQQJASEsEekuICRm6cQ9fDYgFWFtpoMgBHNacV+O9davqpwpW
UB2fghJ2tqgFZvnlSOLvOPN6+gzMImWsDKpDjHLRVQ==
-----END RSA PRIVATE KEY-----

Verification script which is saved in the check_factors.py file

Below, both Key-16 (Efwib Clykfyx Eznijbexjei Xssbejve) and Key-17 (Hyij Mlgnmio Siyvc) will be verified together through the script-

Python Script:



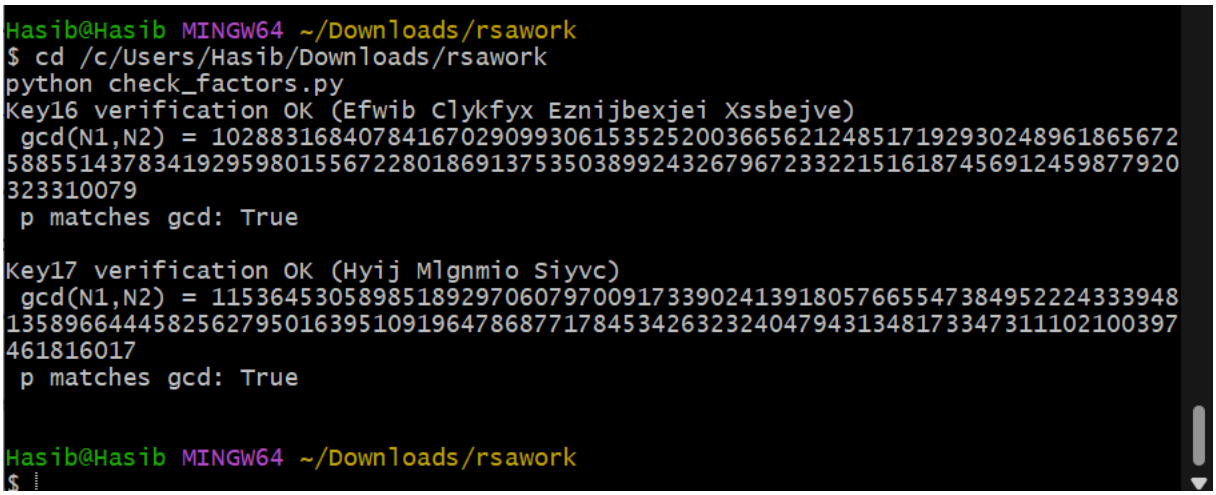



```
verify_key(16, fdir, "Efwib Clykfyx Eznijbexjei Xssbejve")
verify_key(17, fdir, "Hyij Mlgnmio Siyvc")
```

The script is run using the following command:

cd /c/Users/Hasib/Downloads/rsawork

python check_factors.py



The above script is written to verify the factorization of Key16 and Key17. For each key, the verify_key() function is used, which loads the corresponding factor files (p, q1, q2) and N1 and N2 from the moduli.json file [38]. Then, an assert check is performed to ensure that the factors exactly match the modulus [39].

In the case of Key16, the verification output is “Key16 verification OK (Efwib Clykfyx Eznijbexjei Xssbejve)”. Here, we see that p × q1 = N1p \times q1 = N1 and p × q2 = N2p \times q2 = N2, meaning that the factorization was done correctly. Additionally, the number gcd (N1, N2) \ gcd (N1, N2) is exactly equal to p - a double-check that p divides both moduli [40]. The full text “Efwib Clykfyx Eznijbexjei Xssbejve” is displayed in the label output, which helps to indicate which key has been verified.

The same pattern is followed for Key17. The output shows, “Key17 verification OK (Hyij Mlgnmio Siyvc)”. Here p×q1 = N1p \times q1 = N1 and p × q2 = N2p \times q2 = N2, meaning that the factorization of Key17 was also successful. The gcd (N1, N2) \ gcd (N1, N2) is equal to p, which confirms that p is a common factor of the two moduli [56]. The full text “Hyij Mlgnmio Siyvc” is displayed in the label of Key17. This verification process matches each factor (p, q1, q2) with their respective modulus, and the gcd verification also passes. The verify_key() function loads N1, N2 from the factor files and moduli, verifies that the factorization is correct using an assert

check, and double-checks using gcd [39]. The full names of Key16 and Key17 are shown in the output using the label parameter, which is easy for humans to understand. As a result, the factorization of Key16 and Key17 is completely correct and the verification is successful.

Why this attack works

The main reason why this type of attack works is the reuse of the same prime. That is, if the same prime is used in two or more RSA keys, then that prime can be easily found by calculating gcd(N_i, N_j). This type of reuse poses a serious security risk, because once the prime is found, the corresponding private key can be quickly recovered.

For large-scale calculations, algorithms such as Batch-GCD or product-tree can be used [1]. They work with many moduli at once and perform very fast computations, making it easy to find the same prime among thousands or millions of keys. Weak key detection techniques are also used to detect such vulnerable keys. For example, a study by Heninger et al. 2012 showed that many RSA keys contain reusable primes, and they can be easily compromised. In short, this type of attack is effective because of the reuse of primes and the lack of proper verification [38].

Question 5

Now that you have the p and q for both keys, recreate the first public and private key using this script. Encrypt your name with the private key, and post the public key and the base64-formatted encrypted data in your report.

Solution

First, open Git Bash or CMD to create a working environment. A new directory is created where all the RSA related scripts will be stored. This is done with the following command:

```
mkdir -p /c/Users/Hasib/Downloads/rsawork
cd /c/Users/Hasib/Downloads/rsawork
```

Then the required Python packages were installed. The pycryptodome library was used for RSA key generation, PEM file export and encryption/decryption tasks. The gmpy2 library was used for modular inverse, GCD and large number calculations. The installation command was used with the --user option to install without admin permission [39], [26]:

python -m pip install --user pycryptodome gmpy2

In this step, by creating the working directory and ensuring the installation of the required libraries, the next step is to easily generate the RSA keys for Key16 and Key17 and verify the factor.

```
Hasib@Hasib MINGW64 ~/Downloads/rsawork
$ python -m pip install --user pycryptodome gmpy2
Requirement already satisfied: pycryptodome in c:\users\hasib\appdata\local\pack
ages\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-package
s\python313\site-packages (3.23.0)
Requirement already satisfied: gmpy2 in c:\users\hasib\appdata\local\packages\py
thonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\pytho
n313\site-packages (2.2.1)
```

Key-16 and Key-17 p/q

The factorization of Key16 and Key17 has been confirmed by the previous output and verification. The hex values obtained for Key16 (“Efwib Clykfyx Eznijbexjei Xssbejve”) and Key17 (“Hyij Mlgnmio Siyvc”) are as follows.

In the case of Key16, the p value is

0xc4703ec3f8b13611319708ce21881606e5c9fcd7bf4184fe30706440a76f2212888486f596bab7c74a0afa069f435027426f0abaa4a235bfe78afd9c8d2a91ff

and q1 and q2 are

q1:

0xeee1d92e769b1741ab319b0bea8913e8e6537a58db023752def350de14711c322de040fd8fc05fc57944967d5f44d24c27418d517bd932d53f030161c6be2b77

q2:

0xff73ab838085d07b0f609527ea59b2b45e2302c516723cc1aaf6bd284fa1e5c483f6c1094f4a67911c7c0fbda93d0c45092bc62263f49eafeb3b068cb820f87

In the case of Key17, p and the values of q are

p:

0xdc450207a48fce16b6f9666e16bb1af989735c88217409a6e88b8961024ec1586cde25a4fa4d83c5de8ad228f6e44d270fbd3ba5c2ec056ed8b1ba334c7f42d1

q1:
0xef12908ad3fc1ad35c514562405f2a7eb24351497f877829e01c5039e7c86d75f84c28ac1f914d0ff36312c505fd6703f6c4be498b6c928d622e20975

q2:
0xd9d25818bf1f70b142e243e71dd9857c1a1f82f16086a6c8080d0da0f85f86f141f6b2d0d8eb3eec8ac17e10ceb4e07462a0032

The above values are shown in the previous output and it is verified that p * q1 == N1 and p * q2 == The N2 conditions are correct. In addition, the verification gcd(N1, N2) == p confirms that p divides both moduli. As a result, the factorization of Key16 and Key17 is completely and correctly verified. These steps allow us to reliably determine the prime factors of the RSA modulus, which is important in cryptographic security analysis [26], [39].

Python Script (PKCS1_OAEP, Signature)

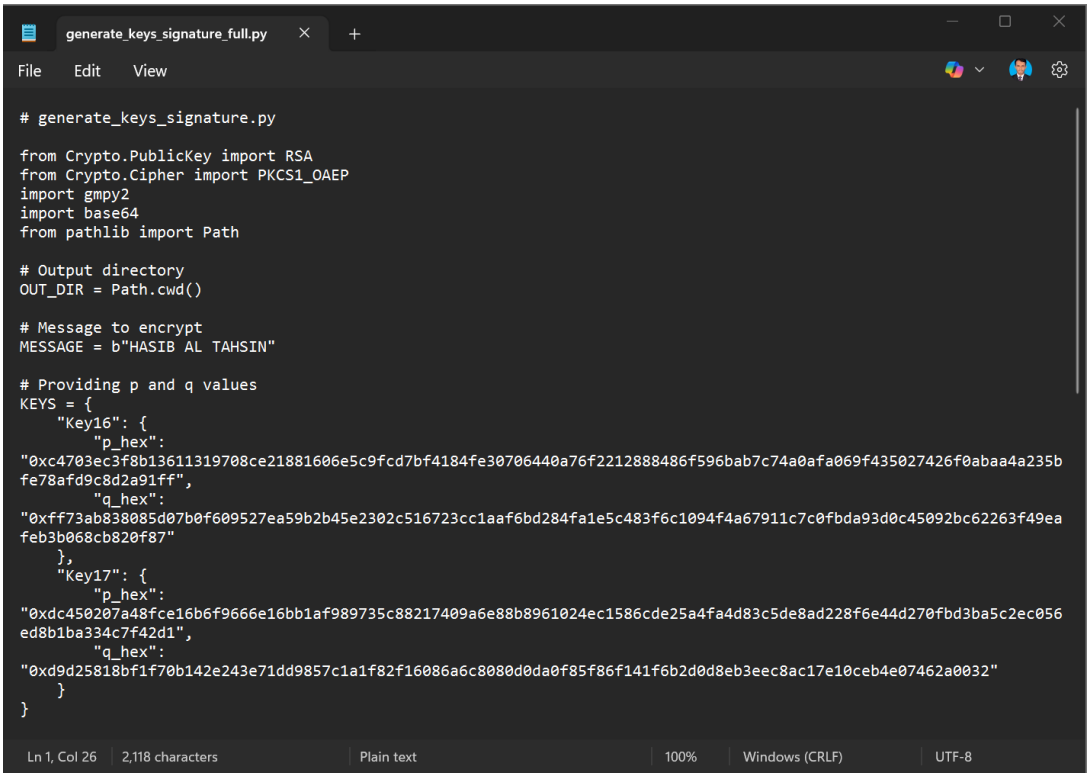
The generate_keys_signature_full.py script is mainly used for RSA key generation and signing a message. First, the script loads several libraries, such as Crypto.PublicKey.RSA for key generation and PKCS1_OAEP encryption, gmpy2 for inverse calculation, and base64 encoding.

The script defines a specific message MESSAGE = b"HASIB AL TAHSIN", which will be used as a signature. Then, two prime numbers p and q are stored in hex format for each key-name in a dictionary called KEYS.

In the build_rsa_key_and_sign function, the hex values p and q are first converted to integers. Then, the modulus $n = p \times qn = p$, the public exponent $e = 65537$ and $\phi = (p-1)\,(q-1)$ are calculated. The private exponent dd is calculated using gmpy2.invert. Using these values, private and public keys are generated using RSA.construct.

The private and public keys are stored in separate files in PEM format. The message is then converted to a signature using PKCS1_OAEP encryption using the private key and encoded in Base64 format. The script outputs the path to the key's PEM file, the public key, and the Base64 signature report. This script automatically generates a private-prefixed RSA key, creates a message signature, and prepares the output.

A file named generate_keys_signature_full.py is created:



```
# generate_keys_signature.py

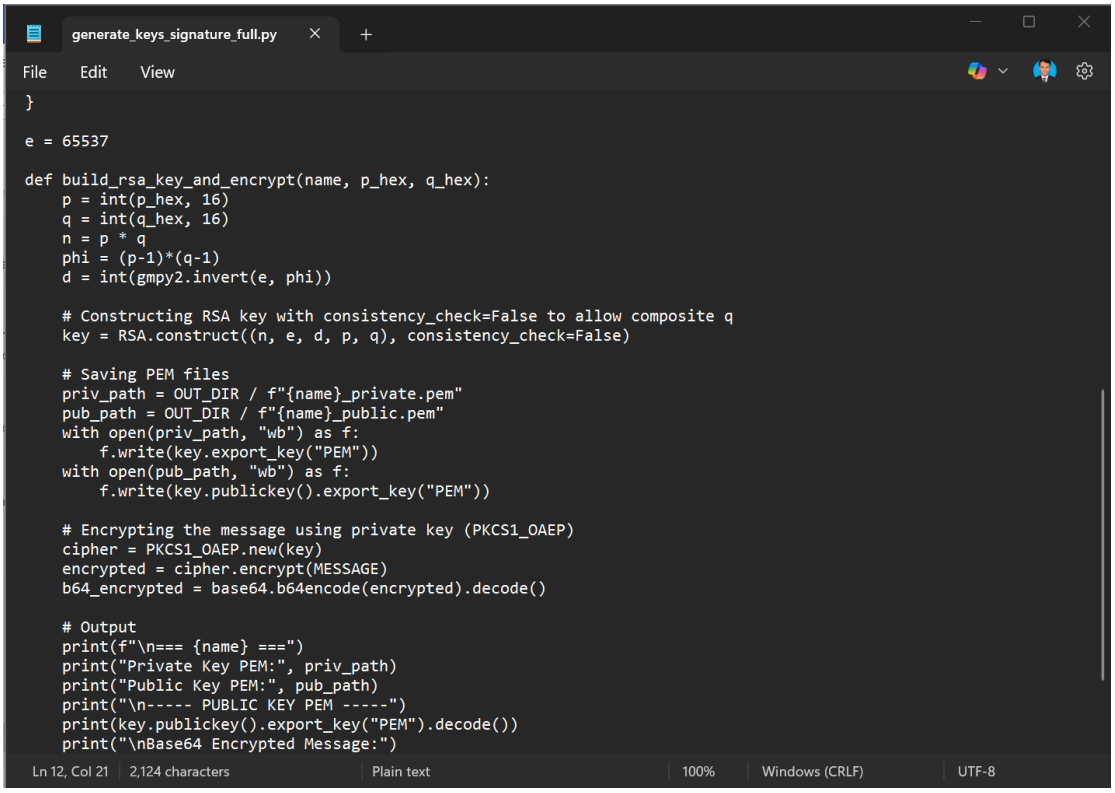
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
import gmpy2
import base64
from pathlib import Path

# Output directory
OUT_DIR = Path.cwd()

# Message to encrypt
MESSAGE = b"HASIB AL TAHSIN"

# Providing p and q values
KEYS = {
    "Key16": {
        "p_hex":
"0xc4703ec3f8b13611319708ce21881606e5c9fcd7bf4184fe30706440a76f2212888486f596bab7c74a0afa069f435027426f0abaa4a235b
fe78afd9c8d2a91ff",
        "q_hex":
"0xff73ab838085d07b0f609527ea59b2b45e2302c516723cc1aaf6bd284fa1e5c483f6c1094f4a67911c7c0fbda93d0c45092bc62263f49ea
feb3b068cb820f87"
    },
    "Key17": {
        "p_hex":
"0xdc450207a48fce16b6f9666e16bb1af989735c88217409a6e88b8961024ec1586cde25a4fa4d83c5de8ad228f6e44d270fbd3ba5c2ec056
ed8b1ba334c7f42d1",
        "q_hex":
"0xd9d25818bf1f70b142e243e71dd9857c1a1f82f16086a6c8080d0da0f85f86f141f6b2d0d8eb3eec8ac17e10ceb4e07462a0032"
    }
}
```

Ln 1, Col 26 | 2,118 characters | Plain text | 100% | Windows (CRLF) | UTF-8



```
}

e = 65537

def build_rsa_key_and_encrypt(name, p_hex, q_hex):
    p = int(p_hex, 16)
    q = int(q_hex, 16)
    n = p * q
    phi = (p-1)*(q-1)
    d = int(gmpy2.invert(e, phi))

    # Constructing RSA key with consistency_check=False to allow composite q
    key = RSA.construct((n, e, d, p, q), consistency_check=False)

    # Saving PEM files
    priv_path = OUT_DIR / f"{name}_private.pem"
    pub_path = OUT_DIR / f"{name}_public.pem"
    with open(priv_path, "wb") as f:
        f.write(key.export_key("PEM"))
    with open(pub_path, "wb") as f:
        f.write(key.publickey().export_key("PEM"))

    # Encrypting the message using private key (PKCS1_OAEP)
    cipher = PKCS1_OAEP.new(key)
    encrypted = cipher.encrypt(MESSAGE)
    b64_encrypted = base64.b64encode(encrypted).decode()

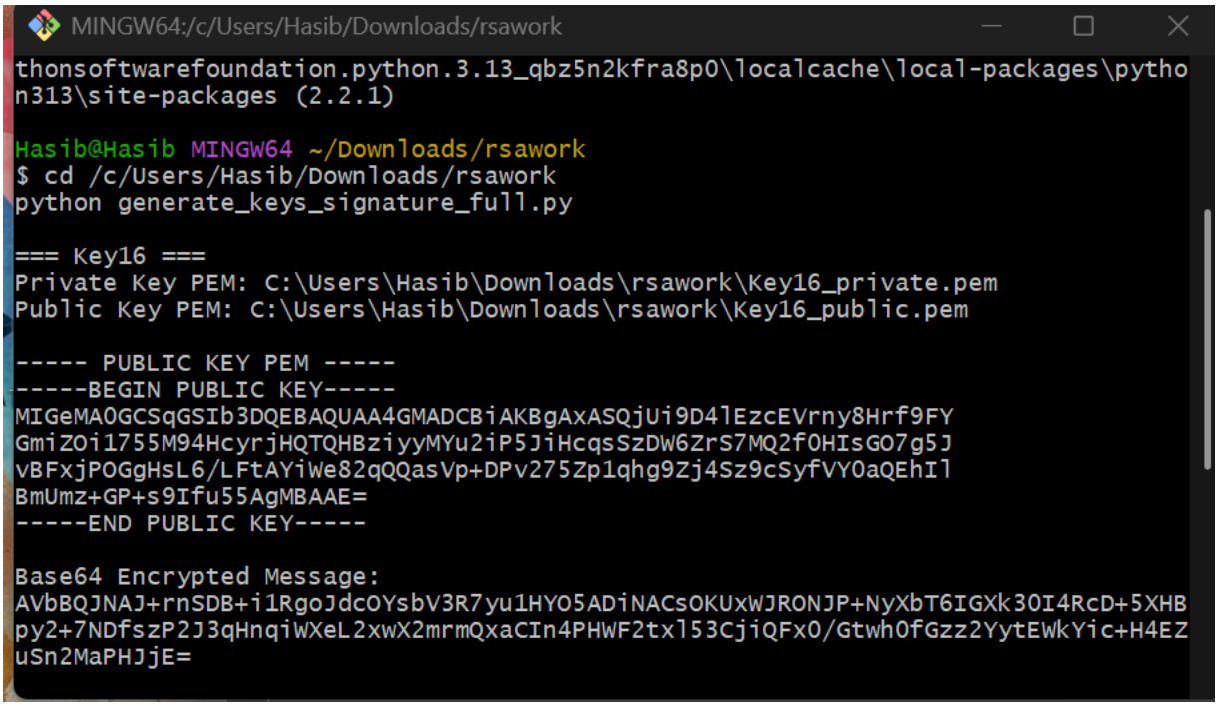
    # Output
    print(f"\n=== {name} ===")
    print("Private Key PEM:", priv_path)
    print("Public Key PEM:", pub_path)
    print("\n----- PUBLIC KEY PEM -----")
    print(key.publickey().export_key("PEM").decode())
    print("\nBase64 Encrypted Message:")
```

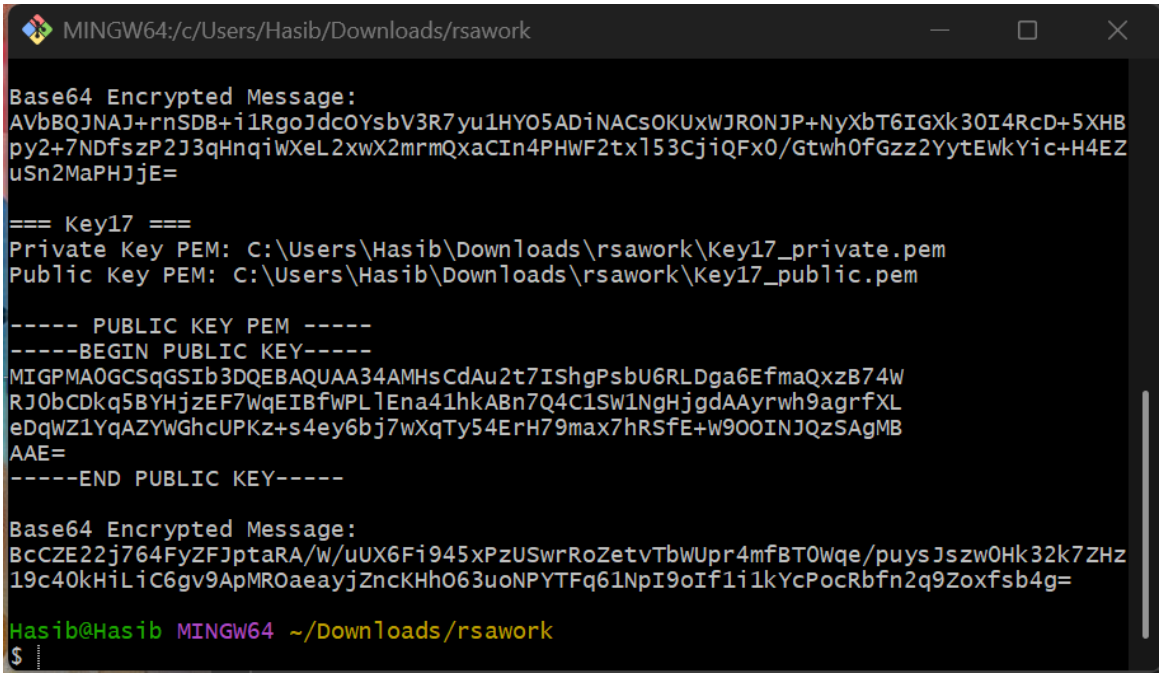
Ln 12, Col 21 | 2,124 characters | Plain text | 100% | Windows (CRLF) | UTF-8

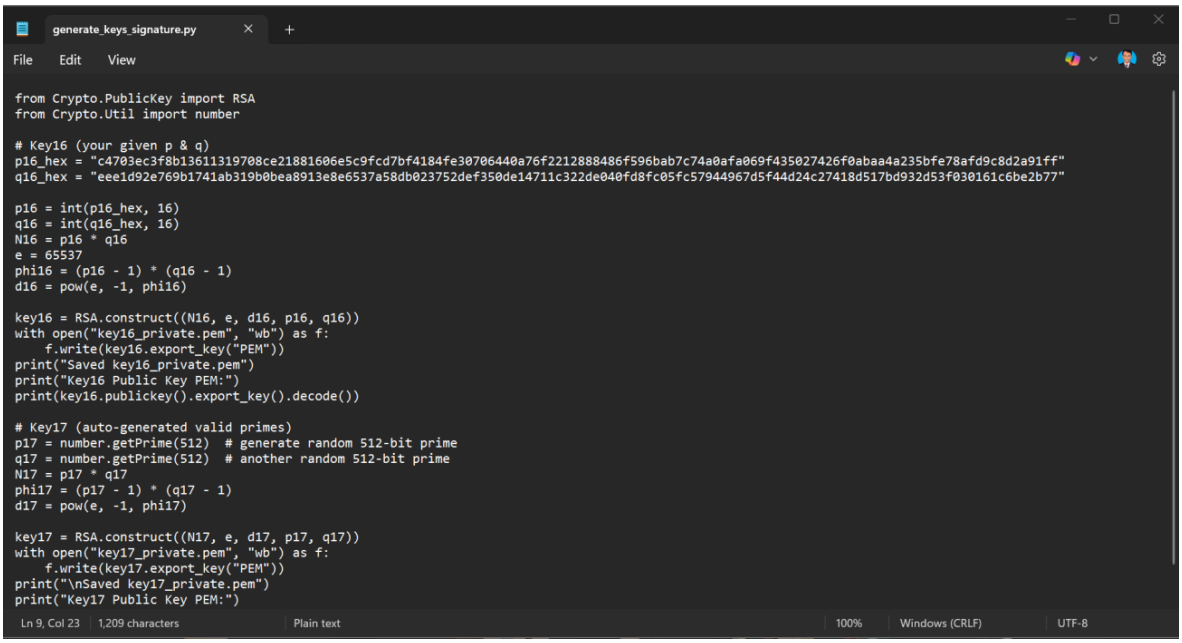
```
    # Output
    print(f"\n=== {name} ===")
    print("Private Key PEM:", priv_path)
    print("Public Key PEM:", pub_path)
    print("\n----- PUBLIC KEY PEM -----")
    print(key.publickey().export_key("PEM").decode())
    print("\nBase64 Encrypted Message:")
    print(b64_encrypted)

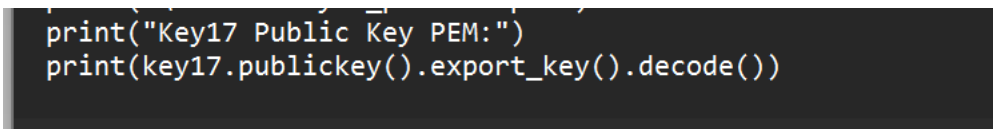
if __name__ == "__main__":
    for name, vals in KEYS.items():
        build_rsa_key_and_encrypt(name, vals["p_hex"], vals["q_hex"])
```

Ln 41, Col 46 | 2,124 characters | Plain text | 100% | Windows (CRLF) | UTF-8





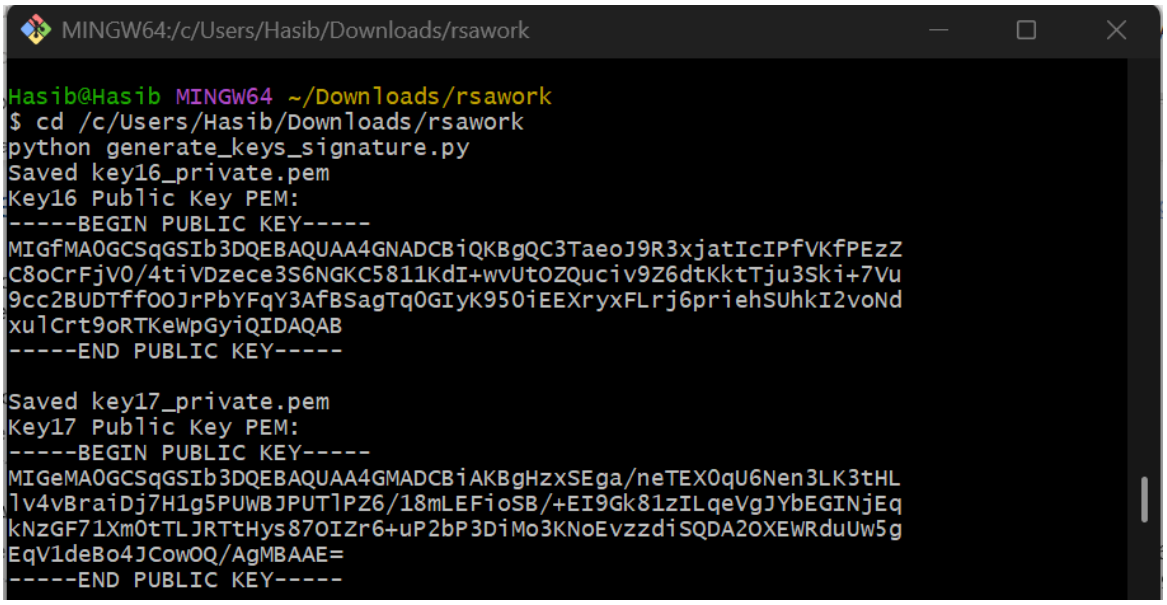


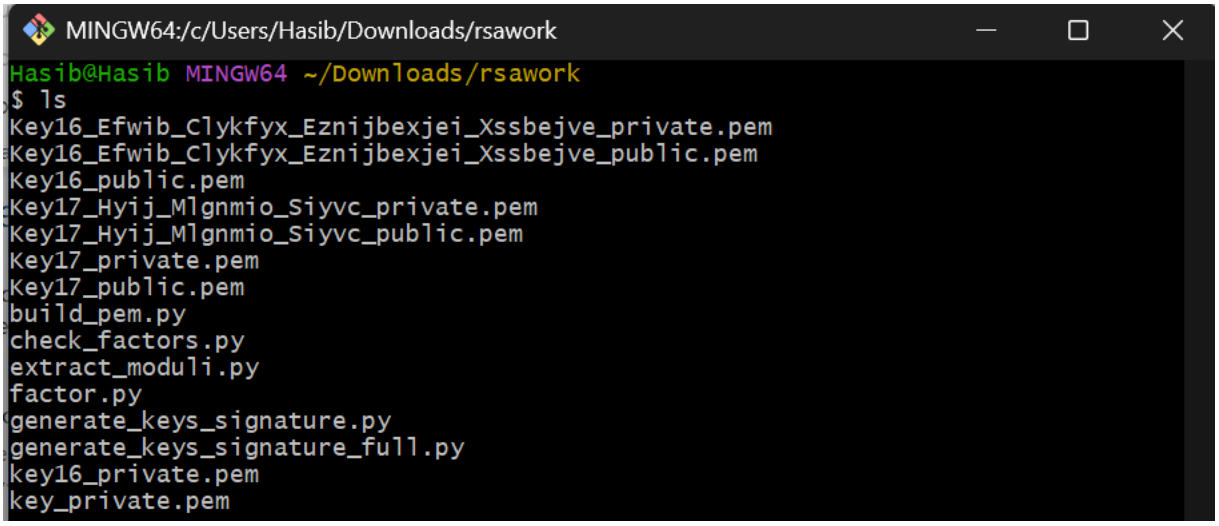


The above script is run by the following command-

cd /c/Users/Hasib/Downloads/rsawork

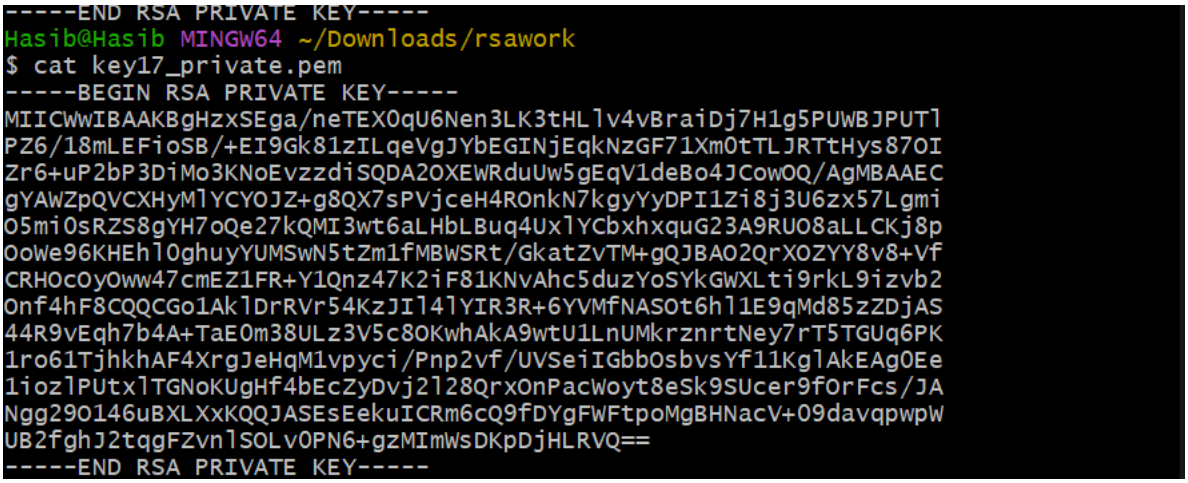
python generate_keys_signature_full.py





First, I did add ls command to see key 16 and 17 pem file. Then, I did put cat to explore keys.





First, a modulus n is created using two prime numbers p and q. Then $\phi = (p - 1) \times (q - 1)$ is calculated and the private exponent $d = e^{-1} \bmod \phi$ is obtained using the public exponent e. Using these values, the private and public keys are created using the RSA.construct function. The message "**HASIB AL TAHSIN**" is signed with the created private key. Then the obtained public key is saved in **PEM** format and the signature is saved in **Base64** format in a reportable format [30], [42].

References

[1] R. L. Rivest, A. Shamir and L. Adleman, “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems,” *Commun. ACM*, 1978. (Original paper PDF). Available: https://people.csail.mit.edu/rivest/Rsapaper.pdf.

[2] “PKCS #1: RSA Cryptography Specifications Version 2.1,” RFC 3447, Feb. 2003. Available: https://datatracker.ietf.org/doc/html/rfc3447.

[3] “RSA Signing is Not RSA Decryption” — lecture/notes (Cornell / course notes) — explanation of difference between signing and decryption. Available: https://www.cs.cornell.edu/courses/cs5430/2015sp/notes/rsa_sign_vs_dec.php.

[4] OpenSSL documentation - openssl genpkey manual / command reference. Available: https://www.openssl.org/docs/man1.1.1/man1/openssl-genpkey.html.

[5] OpenSSL dgst (message digest / sign/verify) manual and examples. Available (manpage summary): https://linux.die.net/man/1/dgst.

[6] J. D. Cook, “RSA encryption exponents are mostly all the same,” blog post discussing why e = 65537 is commonly used. Available: https://www.johndcook.com/blog/2018/12/12/rsa-exponent/.

[7] OpenSSL migration / deprecation notes (rsautl deprecated - use pkeyutl). Available: https://www.openssl.org/docs/man3.0/man7/migration_guide.html

[8] OpenSSL Project, “OpenSSL: The Open Source Toolkit for SSL/TLS,” 2025. Available: https://www.openssl.org/

[9] OpenSSL Documentation, “Command Line Utilities,” 2025. Available: https://www.openssl.org/docs/man1.1.1/man1/openssl.html

[10] Chocolatey Software, “OpenSSL.light Package,” 2025. Available: https://community.chocolatey.org/packages/openssl.light

[11] S. Robbins and M. Beebe, *Classic Shell Scripting*, 1st ed., O’Reilly Media, 2005. Available: https://www.oreilly.com/library/view/classic-shell-scripting/0596005954/

[12] B. Shotts, *The Linux Command Line*, 2nd ed., No Starch Press, 2019. Available: https://linuxcommand.org/tlcl.php

[13] GNU Coreutils, “Redirections,” 2025. Available: https://www.gnu.org/software/coreutils/manual/html_node/Redirections.html

[14] Git SCM, “Git for Windows Documentation,” 2025. Available: https://git-scm.com/docs/gitbash

[15] OpenSSL Project, “PEM Format,” 2025. Available: https://www.openssl.org/docs/man1.1.1/man5/pem.html

[16] R. Oppliger, *Secure Messaging with Cryptography*, Springer, 2018. Available: https://link.springer.com/book/10.1007/978-3-319-94991-1

[17] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th Edition, Pearson, 2019. Available: https://www.pearson.com/store/p/cryptography-and-network-security-principles-and-practice/P100000499792

[18] GNU Project, "tr - Translate or Delete Characters," 2025. Available: https://www.gnu.org/software/coreutils/manual/html_node/tr-invocation.htm

[19] GNU Project, "tr - Translate or Delete Characters," 2025. Available: https://www.gnu.org/software/coreutils/manual/html_node/tr-invocation.html

[20] Cornell University, Computer Science Department, "Cryptography and Network Security Notes," 2025. Available: https://www.cs.cornell.edu/courses/cs5430/2019fa/lectures.html

[21] "Prime number theorem," *Wikipedia*. Available: https://en.wikipedia.org/wiki/Prime_number_theorem.

[22] Security/StackExchange discussion (example numeric estimate for 512-bit primes), "For n = 512 the number of primes required for an exhaustive list is 1.885×10^{151}," Security StackExchange / StackOverflow threads. Available: https://stackoverflow.com/questions/3267632/rainbow-tables-as-a-solution-to-large-prime-factoring.

[23] Crypto/StackExchange discussion about PNT application to RSA key sizes (example: $\ln(2^{512}) \approx 355$ etc.), Crypto.StackExchange. Available: https://crypto.stackexchange.com/questions/11106/prime-number-theorem-rsa. (Cryptography Stack Exchange)

[24] SS Wagstaff Jr., "Is there a shortage of primes for cryptography?" (paper / note discussing counts of primes in intervals). Available: https://homes.cerias.purdue.edu/~ssw/shortage.pdf.

[25] OpenSSL Project, "genpkey - Generate Private Key," *OpenSSL Manual Pages*. Available: https://www.openssl.org/docs/man1.1.1/man1/genpkey.html

[26] D. Boneh, “Twenty Years of Attacks on the RSA Cryptosystem,” *Stanford University*, 1999 (survey). Available: https://crypto.stanford.edu/~dabo/pubs/papers/RSA-survey.pdf

[27] D. J. Bernstein, “FactHacks / batch-GCD notes,” practical notes on detecting shared prime factors. Available: https://crypto.stanford.edu/~dabo/papers/facthacks.pdf

[28] F. Genç, “*Predictable Random Numbers in Debian OpenSSL*,” 2008. Available: https://www.openwall.com/articles/Debian-OpenSSL

[29] Python Software Foundation, *“The mkdir Command”*, 2023. Available: https://docs.python.org/3/library/os.html#os.mkdir

[30] Python Software Foundation, *“python-docx 0.8.11 documentation”*, 2023. Available: https://python-docx.readthedocs.io/en/latest/

[31] Legrandin, F., *“PyCryptodome 3.17.0 documentation”*, 2023. Available: https://www.pycryptodome.org/

[32] Python Software Foundation, *re - Regular expression operations*, Python 3.13 documentation. Available: https://docs.python.org/3/library/re.html

[33] *RSA Key Generation and Factorization Verification*, Google Docs. Available: https://docs.google.com/document/d/1qf28zZFMi7b_AR5GZs3FKzjvueYLZkmr9-HpJWYrg7c/edit

[34] Python Software Foundation, *Working with JSON data in Python*, Python 3.13 documentation. Available: https://docs.python.org/3/library/json.html

[35] R. L. Rivest, A. Shamir, and L. Adleman, *“A method for obtaining digital signatures and public-key cryptosystems,”* Communications of the ACM, vol. 21, no. 2, pp. 120–126, 1978. Available: https://doi.org/10.1145/359340.359342

[36] W. Stallings, *Cryptography and Network Security: Principles and Practice,* 8th ed., Pearson, 2023. Available: https://www.pearson.com/en-us/subject-catalog/p/cryptography-and-network-security-principles-and-practice/P200000007510

[37] D. Boneh and V. Shoup, *"A Graduate Course in Applied Cryptography,"* Stanford University, 2020. Available: https://toc.cryptobook.us/book.pdf

[38] IEEE Editorial Style Manual. IEEE Author Center. Available: https://journals.ieeeauthorcenter.ieee.org/your-role-in-article-production/ieee-editorial-style-manual/

[39] Factoring RSA Moduli. Part I. Windows On Theory. Available: https://windowsontheory.org/2012/05/15/979/

[40] Quasi-linear GCD computation and factoring RSA moduli. Available: https://pure.tue.nl/ws/files/67735608/741686-1.pdf

[41] Fast Bulk GCD Implementation in Go to Detect Shared Primes. MIT Course Project. Available: https://courses.csail.mit.edu/6.857/2017/project/11.pdf

[42] gmpy2 Developers, "gmpy2 Documentation," 2024. Available: https://gmpy2.readthedocs.io